Decision Support System

Decision Support System (DSS) is a real-time decision-making tool where data, models, and software are used in partnership with individuals to generate efficient solutions. It combines numerous data inputs and offers methodological approaches to evaluation, modelling and display of the information to facilitate decision-making in case of the system's challenging issues. DSS facilitates semi-structured and unstructured decision-making and can improve the quality, speed and efficiency of the decisions since new information. Prognoses that would be hard to produce manually are available.

Characteristics of a Decision Support System

- **Interactive Interface:** The graphical user interface is also user friendly hence users can interact with DSS easily in inputting their data and get the desired output.
- **Data Integration:** It gets information from various sources like DBMS, data marts, Data warehouses and even data feeds to have complete data when processing data.
- **Support for Semi-structured and Unstructured Decisions:** DSS on the other hand is intended for usage in cases where the decision-making process is not highly routinized as in the case of traditional management information systems.
- **Analytical Models and Tools:** DSS also has tools for analysing data and making recommendations; these tools range from statistical analysis, forecasting, optimization, and simulation models.
- **Flexibility and Adaptability:** The system can be applied to any type of decision-making environment and is versatile in the sense that it can be modified in exact conformity with the needs of the users or the organization.
- **What-if Analysis:** It helps in what-if analysis where the assumptions or the values of the input variable can be varied to determine the impact of change on the result.
- **Timely and Relevant Information:** DSS thus supplies timely and relevant information that can be used by the decision-makers to respond appropriately to requisite and volatile environments.
- **Support for Group Decision Making:** Most of the available DSS have group support systems, where more than one person is involved in the decision-making process.

Evolution of Decision Support Systems

Overview

Decision support has evolved from fixed, paper-style reports to interactive analytical tools and now to real-time, AI-driven analytics on big data. Each stage appeared because the previous one could not answer managers' growing needs.

Stages of Evolution

Stage	**Period**	**Key Idea**	**Limitation That Led to the Next Stage**
1. Batch Reporting (EDP)	1960s	Mainframes produce printed reports in batches.	Rigid, slow, and no user interaction.

2. MIS (Management Information Systems)	1970s	Fixed, periodic summary reports for managers.	Only answers predefined questions; no “what-if” analysis.
3. DSS (Decision Support Systems)	Late 1970s–1980s	Interactive, model-driven analysis for semi-structured decisions.	Limited to departmental data; no integrated historical view.
4. EIS (Executive Information Systems)	1980s	Dashboards and drill-down summaries for top management.	Data scattered across systems, resulting in inconsistent figures.
5. Data Warehouse and OLAP	1990s	Integrated, historical, subject-oriented data with multidimensional analysis.	Focused on known questions; hidden patterns remained undiscovered.
6. Business Intelligence and Data Mining	2000s	KPIs, scorecards, and pattern discovery using classification, clustering, and association.	Struggled with huge volumes of unstructured and rapidly arriving data.
7. Big Data, Cloud, NoSQL and AI	2010s onward	Real-time analytics, machine learning, cloud platforms, and NoSQL data stores.	Ongoing challenges include governance, privacy, and explainability.

Explanation of Each Stage

1. Batch reporting (1960s)

Computers automated payroll and accounting. Reports were printed on schedule, with no way to ask new questions.

2. MIS (1970s)

Transaction data was summarized into regular reports (monthly sales, stock status). Useful for structured decisions but inflexible.

3. DSS (late 1970s-1980s)

Introduced the idea of a model base, a database and a user interface working together. Managers could run what-if and sensitivity analysis for semi-structured problems such as pricing or budgeting.

4. EIS (1980s)

Designed for senior executives, with graphical dashboards, exception reporting and drill-down. Exposed the problem that data lived in separate systems and did not agree.

5. Data Warehouse and OLAP (1990s)

The warehouse combined data from many sources into a single subject-oriented, integrated, time-variant, non-volatile store (Inmon). OLAP allowed slicing, dicing, roll-up and drill-down on cubes. This is the core of modern decision support.

6. BI and Data Mining (2000s)

Self-service dashboards and tools such as Power BI and Tableau emerged. Data mining moved from "what happened?" to "why?" and "what will happen?".

7. Big Data, Cloud, NoSQL and AI (2010s onward)

Cloud warehouses (Redshift, BigQuery, Snowflake), NoSQL databases (MongoDB, Cassandra), streaming data and machine learning enable real-time, predictive and prescriptive analytics.

Purpose of a Decision Support System

Improving Decision Quality: DSS assists in improving the quality of the decision based on information and analysis that is accurate, comprehensive, pertinent, and timely, therefore making better decisions possible.

Handling Complex Problems: This analytic resource is helpful when dealing with assignments that are structured and unstructured since other approaches may not be efficient in handling these issues by the application of analytical and modelling instruments.

Facilitating Rapid Decision Making: DSS facilitates speedy implementation of the decision by first automating the process of data collection and analysis.

Operational Systems vs. Decision Support Systems

Quick Definitions

An **Operational System (OLTP)** runs the day-to-day business by recording transactions as they happen (a sale, a withdrawal, an order). A **Decision Support System (DSS)** analyses accumulated data, usually held in a data warehouse, to help managers make decisions.

Key Differences

Aspect	**Operational System (OLTP)**	**Decision Support System (DSS / DW)**
Purpose	Run daily business operations	Support analysis and decision making
Data type	Current, up-to-date	Historical, spanning years
Data detail	Highly detailed, transaction level	Summarized and aggregated
Data source	Single application or source	Integrated from many sources
Operations	Insert, update, delete, read	Mostly read-only queries
Queries	Simple, predefined, repetitive	Complex, ad hoc, unpredictable
Response time	Milliseconds to seconds	Seconds to minutes (heavy queries)
Users	Clerks, cashiers, tellers (many)	Managers, analysts, executives (few)

Aspect	Operational System (OLTP)	Decision Support System (DSS / DW)
Database design	Normalized (ER model) to avoid redundancy	Denormalized (star or snowflake schema)
Orientation	Application-oriented	Subject-oriented (sales, customer)
Data volatility	Changes constantly	Non-volatile, loaded periodically
Data size	MBs to GBs	GBs to TBs
Transactions	Short, many per second	Long-running, fewer
Performance goal	Transaction throughput	Query speed and analytical power
Availability need	Very high (24x7)	Moderate (batch refresh windows)
Backup and recovery	Critical, frequent	Less frequent, can reload from sources
Example	ATM withdrawal, billing, order entry	Quarterly sales trend, customer segmentation

Supporting Strategic Planning: Strategic support is well provided by DSS since all organisations need to make long-run forecasts and planning and for this, DSS offers tools in the form of scenarios, forecasts, and simulations.

Enhancing Efficiency: DSS reduces the time and efforts needed to amaze the decision information, assemble data and analyze it, thus enhancing organizational productivity at the operation stage.

Encouraging Collaboration: As has been seen many DSSs make it possible to make collaborative decisions this makes it possible for many people to share information and come up with agreed decisions.

Providing Customizable Solutions: DSS is flexible in a way that it can be designed for an organization to suit the variance needs of the users or departments, hence the solutions developed will fit the organization's needs in the particular context.

Reducing Uncertainty: DSS can be categorized to provide complete prognosis data and thereby contribute towards the decrease of risk in the decision-making maximising the decision-maker's confidence level.

Using the DSS Model to Analyze Decision-Making in a Retail Company

1. The DSS Model

A **Decision Support System** has three core components that work together:

Component	Role	Retail example
Database subsystem	Stores the data used for decisions	Sales, inventory, customer, supplier, and promotion data (from the data warehouse)
Model base (model management)	Analytical models that process the data	Demand forecasting, price optimization, market basket analysis, what-if simulation
User interface (dialog)	Lets managers interact with data and models	Dashboards, ad hoc queries, OLAP reports, scenario input screens

A fourth element is the **decision maker** (store manager, category head, CEO), who supplies judgment. The DSS supports the decision; it does not make it.

2. Decision-Making Process (Simon's Model) Applied to Retail

Phase	What happens	Retail example
Intelligence	Identify the problem or opportunity	Dashboard shows electronics sales fell 15% in Q3
Design	Develop possible solutions	Options: cut prices, run a festival promotion, change stock mix
Choice	Evaluate and pick the best option	What-if model compares profit under each option
Implementation and review	Act and monitor results	Launch the chosen promotion and track sales weekly

3. Worked Scenario: "Should We Discount Electronics Before Onam?"

Step 1: Intelligence (data and OLAP)

- Drill-down on the sales cube: Electronics → Mobiles → Kochi region shows the steepest decline.
- Inventory report shows 40% of mobile stock is more than 90 days old.

Step 2: Design (model base)
Alternatives generated:

- A: 10% discount on old stock
- B: 20% discount plus bundle offer
- C: No discount, wait for new launches

Step 3: Choice (what-if analysis)

Option	Expected extra units	Margin per unit	Projected profit	Stock risk
A: 10% off	+300	₹1,800	₹5.4L	Medium

Option	Expected extra units	Margin per unit	Projected profit	Stock risk
B: 20% off + bundle	+700	₹1,000	₹7.0L	Low
C: No discount	+50	₹2,500	₹1.25L	High

The manager picks **Option B**: highest projected profit and lowest stock risk. (Figures are illustrative.)

Step 4: Implementation and feedback

- Run the campaign across Kerala stores.
- Track daily sales against the forecast on the dashboard.
- Results feed back into the database and improve future models.

4. Types of Retail Decisions Supported

Level	Decision type	Example	DSS tool used
Strategic (unstructured)	Where to open a new store	Regional demand and competitor analysis	Scenario and regression models
Tactical (semi-structured)	Seasonal stocking, pricing	Onam inventory plan	Forecasting, what-if analysis
Operational (structured)	Reorder when stock is low	Auto-reorder alerts	Rules and threshold reports

5. Data Flow in the Retail DSS

POS, ERP, Web orders, Supplier data

↓ ETL

Data Warehouse (database subsystem)

↓

Model base (forecasting, pricing, what-if)

↓

User interface (dashboards, OLAP, reports)

↓

Manager decides → action → results feed back

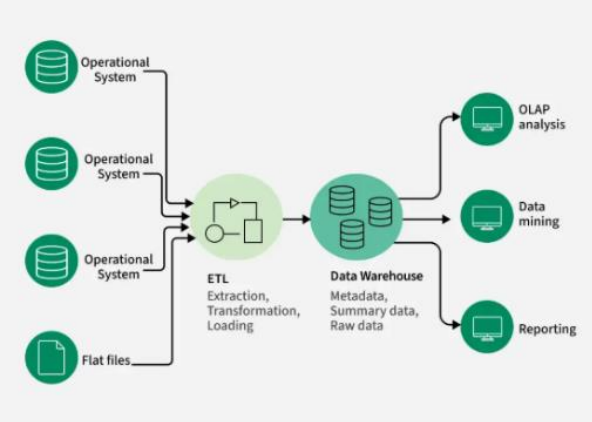
6. Benefits Observed

- Faster, **fact-based decisions** instead of guesswork
- Better **inventory control** (less overstock and stock-outs)
- Improved **pricing and promotion** effectiveness
- Ability to test scenarios **before** committing money

- Consistent figures across departments (one version of the truth)

Data Warehousing

Data warehousing is the process of collecting, integrating, storing and managing data from multiple sources in a central repository. It enables organisations to organise large volumes of current and historical data for efficient querying, analysis and reporting.



Data Warehouse Architecture

The main goal of data warehousing is to support decision-making by providing clean, consistent and timely access to data. It ensures fast data retrieval even when working with massive datasets.

Need for Data Warehousing

- **Handling Large Data Volumes**: Traditional databases store limited data (MBs to GBs), while data warehouses are built to handle huge datasets (up to TBs), making it easier to store and analyze long-term historical data.
- **Enhanced Analytics**: Databases handle transactions; data warehouses are optimized for complex analysis and historical insights.
- **Centralized Data Storage**: A data warehouse combines data from multiple sources, giving a single, unified view for better decision-making.
- **Trend Analysis**: By storing historical data, a data warehouse allows businesses to analyze trends over time, enabling them to make strategic decisions based on past performance and predict future outcomes.
- **Business Intelligence Support:** Data warehouses work with BI tools to give quick access to insights, helping in data-driven decisions and improving efficiency.

Components

- **Data Sources**: These are the various operational systems, databases and external data feeds that provide raw data to be stored in the warehouse.
- **ETL (Extract, Transform, Load) Process**: The ETL process is responsible for extracting data from different sources, transforming it into a suitable format and loading it into the data warehouse.

- **Data Warehouse Database**: This is the central repository where cleaned and transformed data is stored. It is typically organized in a multidimensional format for efficient querying and reporting.
- **Metadata**: Metadata describes the structure, source and usage of data within the warehouse, making it easier for users and systems to understand and work with the data.
- **Data Marts**: These are smaller, more focused data repositories derived from the data warehouse, designed to meet the needs of specific business departments or functions.
- **OLAP (Online Analytical Processing) Tools**: OLAP tools allow users to analyze data in multiple dimensions, providing deeper insights and supporting complex analytical queries.
- **End-User Access Tools**: These are reporting and analysis tools, such as dashboards or Business Intelligence (BI) tools, that enable business users to query the data warehouse and generate reports.

Features of Data Warehousing

Definition

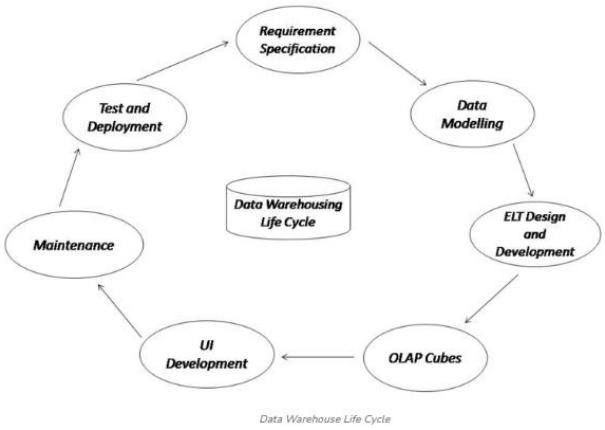
A **data warehouse** is a centralized repository that stores integrated, historical data from multiple sources, designed for **analysis and decision making** rather than daily transactions. According to **Bill Inmon**, it is *subject-oriented, integrated, time-variant, and non-volatile*.

Feature	**Meaning**	**Retail example**
Subject-oriented	Organized around key business subjects (sales, customers, products), not around applications	A "Sales" subject area combines data from POS, online store, and returns
Integrated	Data from different sources is cleaned and made consistent in naming, formats, units and codes	"M/F", "Male/Female", "1/0" are all converted to a single standard
Time-variant	Stores historical data with a time element, so trends over years can be analysed	Sales kept for 5-10 years, each row linked to a date
Non-volatile	Data is loaded and read but not updated or deleted in place	Last year's sales figures remain unchanged once loaded

Data warehouse development life cycle

Data Warehousing is a flow process used to gather and handle structured and unstructured data from multiple sources into a centralized repository to operate actionable business decisions. With all of your data in one place, it becomes easier to perform analysis, reporting and discover meaningful insights at completely different combination levels. A data warehouse setting includes extraction, transformation, and loading (ELT) resolution, an online analytical processing (OLAP) engine, consumer analysis tools, and different applications that manage the method of gathering data and

delivering it to business. The term data warehouse life-cycle is used to indicate the steps a data warehouse system goes through between when it is built. The following is the Life-cycle of Data Warehousing:



Data Warehouse Life Cycle

- **Requirement Specification:** It is the first step in the development of the Data Warehouse and is done by business analysts. In this step, Business Analysts prepare business requirement specification documents. More than 50% of requirements are collected from the client side and it takes 3-4 months to collect all the requirements. After the requirements are gathered, the data modeler starts recognizing the dimensions, facts & combinations based on the requirements. We can say that this is an overall blueprint of the data warehouse. But, this phase is more about determining business needs and placing them in the data warehouse.
- **Data Modelling:** This is the second step in the development of the Data Warehouse. Data Modelling is the process of visualizing data distribution and designing databases by fulfilling the requirements to transform the data into a format that can be stored in the data warehouse. For example, whenever we start building a house, we put all the things in the correct position as specified in the blueprint. That's what data modeling is for data warehouses. Data modelling helps to organize data, creates connections between data sets, and it's useful for establishing data compliance and its security that line up with data warehousing goals. It is the most complex phase of data warehouse development. And, there are many data modelling techniques that businesses use for warehouse design. Data modelling typically takes place at the data mart level and branches out in a data warehouse. It's the logic of how the data is stored concerning other data. There are three data models for data warehouses:
    - Star Schema
    - Snowflake Schema
    - Galaxy Schema.
- **ELT Design and Development:** This is the third step in the development of the Data Warehouse. ETL or Extract, Transfer, Load tool may extract data from various source systems and store it in a data lake. An ETL process can extract the data from the lake, after that transform it and load it into a data warehouse for reporting. For optimal speeds, good visualization, and the ability to build easy, replicable, and consistent data pipelines between all of the existing architecture and the new data warehouse, we need ELT tools. This is where ETL tools like SAS Data Management, IBM Information Server, Hive, etc. come into the

picture. A good ETL process can be helpful in constructing a simple yet functional data warehouse that's valuable throughout every layer of the organization.

- **OLAP Cubes:** This is the fourth step in the development of the Data Warehouse. An OLAP cube, also known as a multidimensional cube or hypercube, is a data structure that allows fast analysis of data according to the multiple dimensions that define a business problem. A data warehouse would extract information from multiple data sources and formats like text files, excel sheets, multimedia files, etc. The extracted data is cleaned and transformed and is loaded into an OLAP server (or OLAP cube) where information is pre-processed in advance for further analysis. Usually, data operations and analysis are performed using a simple spreadsheet, where data values are arranged in row and column format. This is ideal for two-dimensional data. However, OLAP contains multidimensional data, with data typically obtained from different and unrelated sources. Employing a spreadsheet isn't an optimum choice. The cube will store and analyze multidimensional data in a logical and orderly manner. Now, data warehouses are now offered as a fully built product that is configurable and capable of staging multiple types of data. OLAP cubes are becoming outdated as OLAP cubes can't deliver real-time analysis and reporting, as businesses are now expecting something with high performance.
- **UI Development:** This is the fifth step in the development of the Data Warehouse. So far, the processes discussed have taken place at the backend. There is a need for a user interface for how the user and a computer system interact, in particular the use of input devices and software, to immediately access the data warehouse for analysis and generating reports. The main aim of a UI is to enable a user to effectively manage a device or machine they're interacting with. There are plenty of tools in the market that helps with UI development. BI tools like Tableau or PowerBI for those using BigQuery are great choices.
- **Maintenance:** This is the sixth step in the development of the Data Warehouse. In this phase, we can update or make changes to the schema and data warehouse's application domain or requirements. Data warehouse maintenance systems must provide means to keep track of schema modifications as well, for instance, modifications. At the schema level, we can perform operations for the Insertion, and change dimensions and categories. Changes are, for example, adding or deleting user-defined attributes.
- **Test and Deployment:** This is often the ultimate step in the Data Warehouse development cycle. Businesses and organizations test data warehouses to ensure whether the required business problems are implemented successfully or not. The warehouse testing involves the scrutiny of enormous volumes of data. Data that has to be compared comes from heterogeneous data sources like relational databases, flat files, operational data, etc. The overall data warehouse project testing phases include: Data completeness, Data Transformation, Data is loaded by means of ETL tools, Data integrity, etc. After testing the data warehouse, we deployed it so that users could immediately access the data and perform analysis. Basically, in this phase, the data warehouse is turned on and lets the user take the benefit of it. At the time of data warehouse deployment, most of its functions are implemented. The data warehouses can be deployed at their own data center or on the cloud.

Designing a Sales Analysis System Using a Data Warehouse

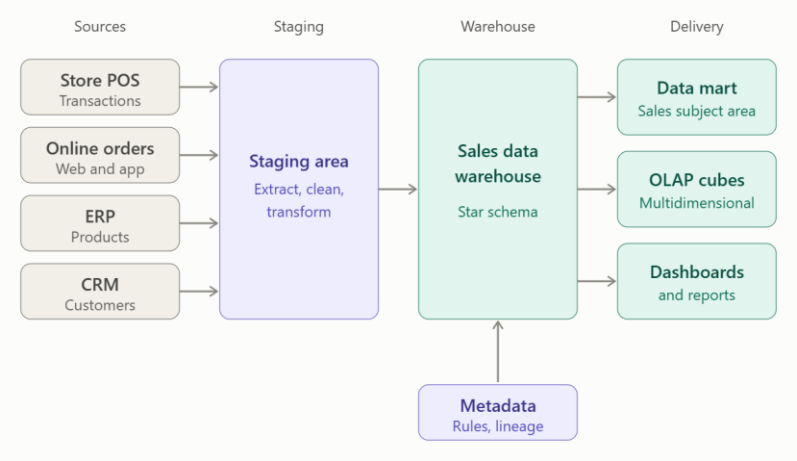
1. Business Requirement

A retail chain with stores in Kerala, Tamil Nadu and Karnataka, plus an online store, wants to analyse sales to answer:

- Which products and categories sell best, and where?
- How do sales vary by month, quarter, and festival season?
- Which customer segments bring the most revenue?
- Which stores or regions are under-performing?

Daily operational systems (POS, e-commerce) cannot answer these quickly, so a **sales data warehouse** is built.

2. Architecture



Layer	**Role**
Source layer	POS, online orders, ERP, CRM
Staging (ETL)	Extract, clean (remove duplicates, fix nulls), standardize, load
Warehouse storage	Integrated, historical star schema
Delivery layer	Data marts, OLAP cubes, dashboards for users
Metadata	Definitions, mappings, load logs

3. Data Model: Star Schema

Granularity: one row per **product, per store, per customer, per day** (transaction line level).

Dim_Date

|

Dim_Product -- Fact_Sales -- Dim_Store

|

Dim_Customer

Fact table

sql

```
Fact_Sales (
  sales_id       INT PRIMARY KEY,
  date_key       INT,      -- FK to Dim_Date
  product_key    INT,      -- FK to Dim_Product
  store_key      INT,      -- FK to Dim_Store
  customer_key   INT,      -- FK to Dim_Customer
  units_sold     INT,      -- measure
  unit_price     DECIMAL(10,2),
  discount       DECIMAL(10,2),
  sales_amount   DECIMAL(12,2),  -- measure
  profit         DECIMAL(12,2)   -- measure
)
```

Dimension tables

Dimension	**Attributes**	**Hierarchy**
Dim_Date	date_key, full_date, day, month, quarter, year, is_festival	Day → Month → Quarter → Year
Dim_Product	product_key, name, brand, category, sub_category	Product → Sub-category → Category
Dim_Store	store_key, store_name, city, state, region, channel (online/offline)	Store → City → State → Region
Dim_Customer	customer_key, name, gender, age_group, city, segment	Customer → Segment

4. ETL Process

1. **Extract:** pull the day's sales from POS and online systems each night.
2. **Transform:**
    - Standardize dates (10-01-25 → 2025-01-10)
    - Map product codes to product_key
    - Remove duplicate transactions
    - Calculate sales_amount = units × price - discount

3. **Load:** insert into dimensions first, then Fact_Sales.
4. **Validate:** compare warehouse totals with source totals.

5. Sample Data

Fact_Sales (sample)

date_key	product_key	store_key	units	sales_amount (₹)
20250110	P01	S01	2	1,000
20250110	P02	S02	1	2,500
20250111	P01	S02	3	1,500

6. Sales Analysis Queries

Sales by category and state per year

sql

```
SELECT p.category, s.state, d.year, SUM(f.sales_amount) AS total_sales
FROM Fact_Sales f
JOIN Dim_Product p ON f.product_key = p.product_key
JOIN Dim_Store   s ON f.store_key   = s.store_key
JOIN Dim_Date    d ON f.date_key    = d.date_key
GROUP BY p.category, s.state, d.year;
```

Top 5 products in Q4

sql

```
SELECT p.name, SUM(f.units_sold) AS units
FROM Fact_Sales f
JOIN Dim_Product p ON f.product_key = p.product_key
JOIN Dim_Date d ON f.date_key = d.date_key
WHERE d.quarter = 4
GROUP BY p.name
ORDER BY units DESC
LIMIT 5;
```

7. OLAP Operations on the System

Operation	Example
Roll-up	Daily sales → monthly → quarterly → yearly
Drill-down	Kerala sales → Ernakulam → Kochi store
Slice	Sales for year = 2025 only
Dice	Electronics and Clothing, in Kerala and Karnataka, 2024-25
Pivot	Swap rows (products) and columns (regions)

8. Output to Users

User	Output	Decision supported
Store manager	Daily sales dashboard	Staffing, local stock
Category head	Product performance report	Reorder and discounting
Marketing	Customer segment analysis	Targeted campaigns
Executives	Regional and yearly KPI summary	Expansion and strategy

9. Benefits of the Design

- **One version of the truth:** all departments see the same sales numbers.
- **Fast analysis:** star schema and aggregation avoid heavy operational queries.
- **Historical trends:** years of data support forecasting.
- **No load on operational systems:** POS stays fast.
- **Scalable:** new subjects (inventory, returns) can be added as new fact tables sharing the same dimensions (conformed dimensions).

Data Flow Through the Data Warehousing Lifecycle (Example)

1. Overview

The data warehousing lifecycle covers both the **project phases** (plan → build → maintain) and the **data movement** (source → staging → warehouse → delivery → user). This example follows one retail sale from the moment it happens to the moment it appears in a manager's report.

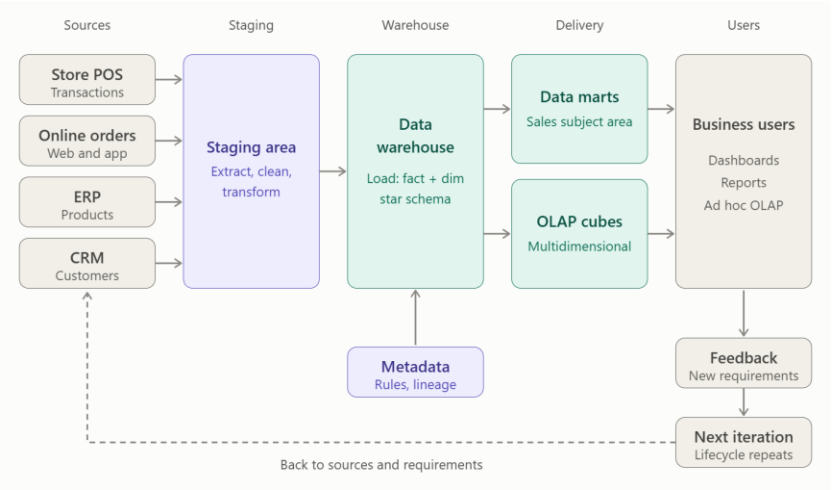
Scenario: A retail chain (stores in Kerala, Tamil Nadu, Karnataka, plus an online shop) builds a sales data warehouse.

2. Lifecycle Phases and the Data Flow at Each Stage

Phase	Activity	Data flow / output
1. Planning	Define scope: "Sales analysis for all stores and online"	Project plan, source list (POS, online orders, ERP, CRM)

Phase	Activity	Data flow / output
2. Requirements	Interview managers; define KPIs (net sales, units, profit, growth %)	Business requirements document, KPI definitions
3. Design	Dimensional modelling, ETL mapping, architecture	Star schema (Fact_Sales, Dim_Date, Dim_Product, Dim_Store, Dim_Customer), source-to-target mapping
4. Build / Development	Create tables, write ETL jobs, build cubes and dashboards	Working staging, warehouse, and delivery layers
5. Testing and Deployment	Validate totals, user acceptance testing, go live	Verified data, signed-off system
6. Maintenance and Growth	Daily loads, monitoring, tuning, add new subject areas	Refreshed data, new fact tables (inventory, returns)

3. End-to-End Data Flow Diagram



4. Tracing One Transaction Through the Flow

Event: On 10 January 2025, a customer buys 2 phone covers at ₹500 each at the Kochi store.

Step 1: Source (operational system)

POS record:

txn_id	date	prod_code	store	cust_id	qty	price	disc
88231	10-01-25	PC-17	KCH01	C4501	2	500	0

Step 2: Extract

The nightly job at 1:00 AM copies the day's POS, online, and ERP changes into the staging area. The operational system is only lightly touched.

Step 3: Staging (clean and transform)

Task	Before	After
Date standardization	10-01-25	2025-01-10 (date_key 20250110)
Code mapping	PC-17	product_key P017 (Accessories → Covers)
Store mapping	KCH01	store_key S01 (Kochi, Kerala, South)
Duplicate removal	txn 88231 appears twice	one record kept
Derived measure	None	sales_amount = 2 × 500 − 0 = 1000
Null handling	missing gender	set to "Unknown"

Rejected or bad rows go to an **error table** for review.

Step 4: Load into the warehouse

Dimensions are loaded first (new customers, products), then the fact table:

date_key	product_key	store_key	customer_key	units_sold	sales_amount	profit
20250110	P017	S01	C4501	2	1000	300

Step 5: Delivery (data mart and cube)

- The row feeds the **Sales Data Mart**.
- The OLAP cube aggregates it: Accessories → Kerala → January 2025 → total sales updated.

Step 6: User access

A category head opens the dashboard: "January 2025 accessories sales in Kerala = ₹4.2 lakh." Drilling down shows Kochi store and then the covers product line, where this ₹1,000 sale is one small part.

Step 7: Feedback and next iteration

The manager asks, "Can we also see return rates?" This becomes a new requirement, so the lifecycle starts again: design a Fact_Returns table sharing the same dimensions, then build, test, and deploy.

Schema for a Simple Data Warehouse (Staging, Storage, Delivery Layers)**5. Metadata Along the Flow**

Stage	Metadata captured
Source	Source system names, field definitions
Staging	Transformation rules, rejected record counts
Warehouse	Table structures, keys, refresh time
Delivery	KPI definitions, cube hierarchies
Operations	Load log: "10 Jan 2025, 1:00-1:40 AM, 1.2M rows loaded, 3 rejected"

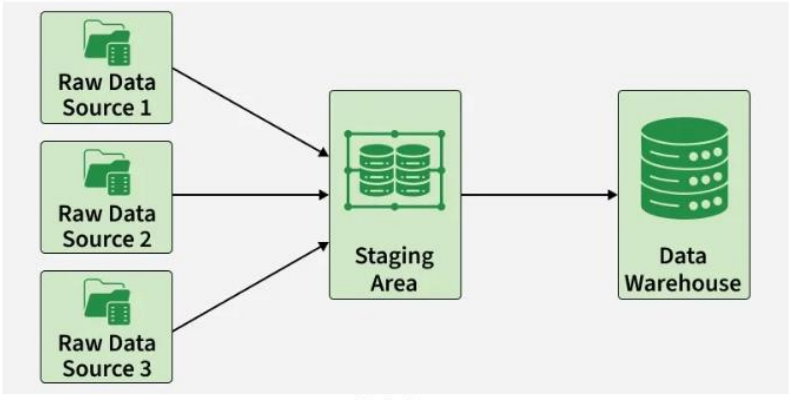
Q:Develop a schema for a simple Data Warehouse using staging, storage, and delivery layers.

PostgreSQL schema for a retail-sales example, in data_warehouse_schema.sql. It uses three schemas, one per layer:

Layer	Schema	Purpose	Design
Staging	stg	Raw landing zone for source extracts	Tables for customers, products and orders. Columns are all text, with no constraints. Audit columns (_batch_id, _source, _loaded_at) are added. Tables are truncated and reloaded each batch.
Storage	dwh	Cleansed, typed, historized single source of truth	Star schema with surrogate keys. dim_date is a standard date dimension. dim_customer and dim_product track history as SCD Type 2 (valid_from, valid_to, is_current). fact_sales has foreign keys, a CHECK on quantity and a unique key on order and product. Supporting tables are dq_rejects for rows that fail validation and etl_batch_log for audit.
Delivery	mart	Business-ready, read-only layer for BI tools	v_sales_detail is a denormalized view. mv_monthly_sales is a pre-aggregated materialized view. v_customer_ltv shows customer lifetime value.

Data Staging Area in Data Warehouse

A Data Staging Area (also known as a data conduit or data clearing house) is a temporary storage space where raw data from multiple sources is collected, cleaned, transformed and prepared before being loaded into the data warehouse. It acts as a bridge between source systems (like ERP, CRM, or flat files) and the data warehouse, ensuring data quality, schema alignment and processing efficiency.



Staging Area

Importance of Data Staging Area

- **Data Consolidation**: Collects data from multiple sources into one place, making it easier to process uniformly.
- **Enhanced Data Quality**: Cleanses data through deduplication, error correction and standardization before loading.

- **Schema Alignment**: Maps and converts different source formats into a unified schema for the warehouse.
- **Improved Performance**: Handles heavy transformations outside the warehouse to ensure faster analytical queries.
- **Temporary Storage for Batches**: Stores incoming data temporarily so it can be synchronized and loaded in organized batches.
- **Better Data Security**: Provides a controlled zone where sensitive data undergoes ETL with proper security measures.
- **Audit & Traceability**: Maintains logs of all ETL steps, enabling full data lineage and easier troubleshooting.

Design Phase Deliverable in Data Warehousing

The key deliverable of the design phase is the **dimensional data model**, usually a star schema, documented as a schema design. It shows the fact tables, the dimension tables, how they relate, and the grain of each fact table, such as one row per sold line item.

Everything built later depends on this model. The ETL, the delivery layer, and the reports and dashboards are all built to match it.

What the deliverable typically contains

- **Business process and grain:** what is being measured (sales) and at what level of detail.
- **Fact table design:** measures such as quantity, sales amount, and cost amount, plus the foreign keys to the dimensions.
- **Dimension table design:** attributes for date, product, customer, and store, including hierarchies like category > product or year > quarter > month.
- **Keys and relationships:** surrogate keys and how the facts connect to the dimensions.
- **Source-to-target mapping:** which source field feeds which warehouse column, and the transformation rules applied (this is the input for the ETL work).
- **Metadata:** definitions of terms, data types, and business rules.

Data Marts

Fundamentals

Definition. A data mart is a subject-oriented, integrated, department-level subset of an organisation's data. It serves the analytical needs of one business unit, such as sales, marketing, finance, or HR.

Key characteristics

- **Subject-oriented:** it covers one business area.
- **Small and focused:** it holds a limited volume of data, usually from a few sources.
- **Dimensional design:** it is usually a star or snowflake schema.
- **Fast:** simpler schemas and smaller data volumes give quick queries.

- **Owned by a department:** users have more control and a quicker development cycle.
- **Read-oriented:** it is used for analysis and reporting, not for transactions.

Why organisations build data marts

1. Faster query response than querying the full warehouse.
2. Lower cost and shorter build time than an enterprise warehouse.
3. Simpler for business users, who see only the data relevant to them.
4. Easier access control, since sensitive data stays out of the mart.
5. Lighter load on the central warehouse.

Types of data mart

Type	Source of data	Approach	Typical risk
Dependent	The enterprise data warehouse (EDW)	Top-down (Inmon)	High up-front cost and time
Independent	Operational or external sources directly	Bottom-up (Kimball, in its standalone form)	Data silos and inconsistent numbers
Hybrid	EDW plus operational or external sources	Mixed	Direct feeds bypass EDW cleansing

1. Difference between a data warehouse and a data mart

Main difference: scope. A data warehouse is enterprise-wide, integrates many subjects, and serves the whole organisation. A data mart is department-specific and covers a single subject area.

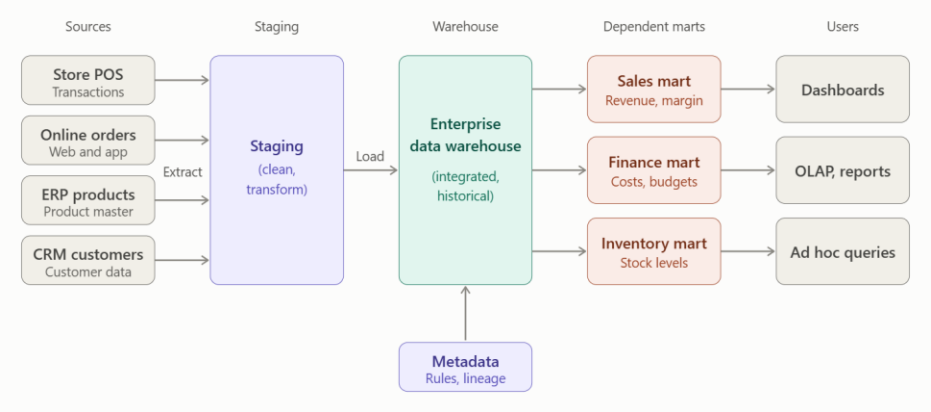
Aspect	Data warehouse	Data mart
Scope	Whole organisation	One department or function
Subject areas	Many	One
Data volume	Very large (TB to PB)	Smaller (GB to low TB)
Sources	Many operational systems	Few sources, or the warehouse itself
Detail level	Detailed, historical data	Often summarised or subject-specific
Build time	Months to years	Weeks to months
Cost	High	Lower
Design	Normalised or dimensional, enterprise model	Dimensional (star or snowflake)

Aspect	Data warehouse	Data mart
Users	Analysts, data scientists, management	Departmental users
Flexibility	Less flexible, changes affect many users	More flexible for local changes

2. Applying dependent data marts to a sales analysis system

Concept. In the dependent approach, the EDW is built first. It is the single integrated repository, and every mart is loaded from it.

Architecture



Design steps

1. **Gather requirements.** Interview the sales team. Typical questions are revenue by region and month, best-selling products, profit margin by category, and customer segments.
2. **Select the business process and grain.** The process is sales, and the grain is one row per sold line item.
3. **Identify dimensions and facts.**
    - Fact table fact_sales with the measures quantity, unit price, discount, sales amount, and cost.
    - Dimensions dim_date, dim_product, dim_customer, and dim_store.
4. **Build the EDW.** Extract from POS, online orders, ERP, and CRM into staging. Cleanse, integrate, and load the data into the warehouse.
5. **Load the sales mart from the EDW.** Take only the sales subset, and refresh it on a schedule such as nightly.
6. **Add aggregates and OLAP cubes.** For example, a monthly sales summary by category. This supports drill-down from year to quarter to month, roll-up, slice, and dice.
7. **Deliver to users.** Provide dashboards, standard reports, and ad hoc analysis.
8. **Govern and secure.** Apply role-based access, data quality checks, and audit logging.

KPIs the sales mart can support

- Total revenue and units sold.
- Gross margin.
- Average order value.
- Year-on-year and month-on-month growth.
- Sales by region, store, channel, and category.
- Customer retention and repeat purchase rate.

Advantages of this design

- Consistent data across all departments.
- One cleansing and integration process to maintain.
- New marts can be added quickly because the warehouse already exists.

Disadvantages

- A large up-front investment.
- A longer wait before the first mart delivers value.
- The marts depend on the warehouse being available and up to date.

3. Why a dependent data mart ensures data consistency

A dependent mart draws all its data from the EDW, which is the single source of truth.

1. **Cleansed once, reused everywhere.** Cleansing, transformation, and business rules are applied once in the ETL into the warehouse. Every mart inherits the same corrected data.
2. **Conformed dimensions.** All marts share the same product, customer, date, and store dimensions. A "customer" or "product category" has the same meaning in every mart.
3. **Common metric definitions.** KPIs such as "net sales" are defined once, so departments do not report different values for the same measure.
4. **Centralised metadata and data lineage.** A change to a source or a rule is made in one place and flows to all marts.
5. **No data silos.** Departments do not extract their own copies directly from operational systems, so there is no divergence between departmental versions.
6. **Single point of governance.** Data quality checks, security, and audit controls are applied centrally.
7. **Consistent history.** The warehouse keeps history (for example, slowly changing dimensions), so every mart sees the same historical records.

Example. Total customer revenue is the same in the sales mart and the finance mart, because both read it from the same warehouse record.

Contrast with independent marts. Each independent mart applies its own rules to the source data. The same measure can then give different results in different marts, which is the "multiple versions of the truth" problem.

4. Hybrid data mart in a retail business (online and offline)

Concept. A hybrid mart takes its core data from the EDW and also loads extra data directly from operational or external sources. It is useful when some data is not yet in the warehouse or needs to be fresher than the warehouse's batch cycle.

Retail scenario: an omnichannel sales mart

Part	Source	Channel
Historical sales, product, and customer master data	EDW (from ERP and CRM)	Offline and online
Store POS transactions	POS system, loaded directly	Offline
Online orders, web clickstream, and cart data	E-commerce platform, loaded directly	Online
Promotions, competitor prices, weather, and market data	External files and feeds	Both

How it works

1. The mart loads the EDW's conformed dimensions (product, customer, date) as its foundation.
2. It adds the latest POS and online order feeds directly, so the data is near real time and ahead of the EDW's next batch.
3. A channel attribute (Store or Online) in the store dimension lets one fact table hold both kinds of sales.
4. A shared customer key, matched through the loyalty ID or email, links a customer's in-store and online purchases.

Analyses this enables

- Compare online and in-store revenue by product category.
- Build a single customer view across both channels.
- Support omnichannel campaigns such as "browse online, buy in store" or "click and collect."
- Track stock and demand across stores and the website.
- Measure the effect of promotions, and of external factors such as weather, on each channel.

Benefits

- Fresher data than a purely dependent mart.

- Broader coverage than the EDW alone, because it can include external data.
- Faster delivery, since the mart does not wait for new sources to be added to the warehouse.

Risks and controls

- Data loaded directly bypasses the EDW's cleansing, so it must be reconciled with the warehouse later.
- Use the same conformed dimensions and key mappings to keep the direct feeds consistent with the EDW data.
- Document the direct feeds in the metadata so lineage is clear.

5. A source from which an independent data mart collects data

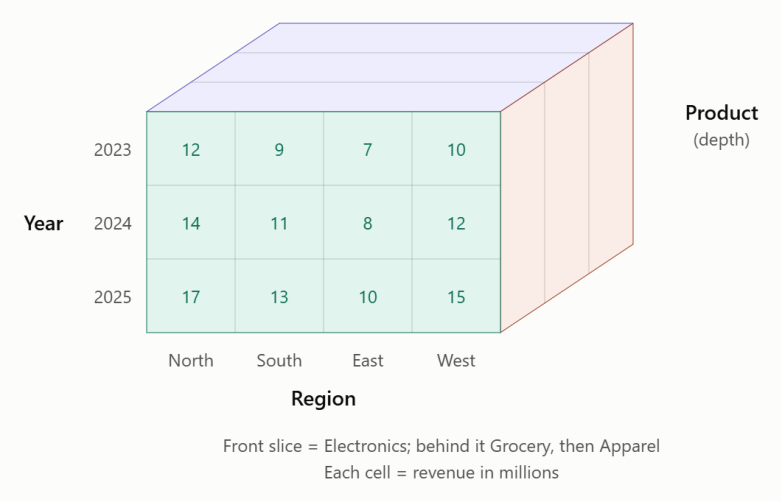
An independent data mart collects data directly from **operational (OLTP) source systems**, with no data warehouse in between. A typical example is the **store POS system**. Other sources include an ERP or CRM system, flat files, and external data feeds.

Characteristics of independent marts

- Quick and cheap to build for one department.
- Each mart has its own ETL and its own business rules.
- Risk of data silos and inconsistent figures across departments.
- Hard to integrate later into an enterprise view.

6. Comparison of the three data mart types

Aspect	**Dependent**	**Independent**	**Hybrid**
Data source	EDW only	Operational sources only	EDW plus operational or external
Consistency	High	Low	Medium to high
Build speed	Slower (needs EDW first)	Fastest	Medium
Cost	High up front	Low	Medium
Data freshness	Depends on the EDW load cycle	Can be fresh	Fresh for direct feeds
Risk	Dependence on the EDW	Silos and duplicated ETL	Direct feeds bypass EDW cleansing
Best suited to	Large enterprises needing one truth	Small teams, quick pilots	Businesses needing fresh and integrated data



OLAP in Data Warehousing

Background: why OLAP exists

A data warehouse stores large volumes of integrated, historical data. Raw tables alone are slow to query and hard for business users to explore. OLAP sits on top of the warehouse (or a data mart) and makes that data fast and easy to analyse.

Position in the architecture

Sources → ETL → Data warehouse → OLAP server (cubes) → Dashboards, reports, ad hoc analysis

OLTP vs OLAP

Aspect	**OLTP (operational)**	**OLAP (analytical)**
Purpose	Run day-to-day transactions	Support analysis and decisions
Operations	Insert, update, delete	Read-heavy queries and aggregation
Data	Current, detailed	Historical, summarised and detailed
Design	Normalised (3NF)	Dimensional (star schema, cubes)
Query type	Simple, short	Complex, multi-dimensional
Records touched per query	Few	Millions
Users	Clerks, customers	Analysts, managers
Response goal	Milliseconds per transaction	Seconds for large aggregations
Example	Recording a sale at the POS	Sales by region and quarter for 3 years

OLAP vs data mining

OLAP	Data mining
User-driven, hypothesis-based ("is Q4 strong in the North?")	Algorithm-driven, discovery-based ("which patterns exist?")
Aggregation and navigation	Classification, clustering, association rules
Answers "what happened and where?"	Answers "why, and what is likely next?"

Definition and purpose of OLAP

Definition. OLAP (Online Analytical Processing) is a category of technology that lets users quickly analyse multidimensional data from many viewpoints. It organises data into cubes of measures and dimensions and provides operations such as roll-up, drill-down, slice, dice, and pivot. The term was introduced by E. F. Codd in 1993.

Purpose

1. Give fast, interactive answers to complex business questions.
2. Summarise large volumes of data at different levels of detail.
3. Let users view the same data by many dimensions (time, product, region, customer).
4. Support trend analysis, comparison, and forecasting.
5. Reduce dependence on IT, since business users can explore the data themselves.
6. Support decision-making, planning, budgeting, and performance monitoring.
7. Provide a single, consistent set of business definitions (for example "net sales") for all users.
8. Support what-if analysis and scenario planning.
9. Reduce the load on operational systems and on the raw warehouse tables.

Main characteristics of OLAP systems

1. **Multidimensional view.** Data is seen as a cube with several dimensions, not as flat tables.
2. **Fast, consistent response.** Pre-computed aggregates give answers in seconds.
3. **Support for hierarchies.** For example, day → month → quarter → year, or store → city → region → country.
4. **Aggregation and summarization.** Measures are summed, averaged, and counted at every level.
5. **Analytical operations.** Roll-up, drill-down, slice, dice, and pivot are available.
6. **Intuitive, interactive interface.** Users work through dashboards, pivot tables, and drag-and-drop tools.
7. **Read-mostly access.** Data is loaded in batches and is not updated by users during analysis.
8. **Large historical data.** It supports comparison over time.

9. **Multi-user and secure.** Many users query at once, with role-based access.
10. **Calculated measures.** Derived values such as margin, growth %, and share are defined once and reused.
11. **Time intelligence.** Built-in functions for year-to-date, same period last year, and rolling averages.
12. **Handling of sparse data.** Cubes store only non-empty cells efficiently, because most combinations (for example a product never sold in a region) have no data.
13. **Dimension-independent operations.** Every dimension is treated equally, so any can be sliced or pivoted.

The FASMI test (Nigel Pendse) is a common summary of OLAP:

Letter	Meaning	Explanation
F	Fast	Most answers return in about 5 seconds
A	Analysis	Handles business and statistical analysis
S	Shared	Multi-user with security
M	Multidimensional	The core requirement
I	Information	Scales to large volumes of data

Codd's rules. Codd's original 12 rules for OLAP include a multidimensional conceptual view, transparency, consistent reporting performance, a client-server architecture, generic dimensionality, dynamic sparse-matrix handling, multi-user support, unrestricted cross-dimensional operations, intuitive data manipulation, flexible reporting, and unlimited dimensions and aggregation levels. FASMI is the shorter test used in practice.

The multidimensional model

Core concepts

Term	Meaning	Retail example
Fact / measure	A numeric value to analyse	Revenue, units sold, cost
Dimension	A perspective for analysis	Date, product, region, customer
Hierarchy	Levels within a dimension	Year → quarter → month → day
Member	One value in a dimension	"North", "2025", "Electronics"
Cell	The measure at one intersection	North, 2024, Electronics = 14
Cube	The full set of cells across dimensions	Revenue by region × year × product
Cuboid	One group-by combination within the cube	Revenue by region only

Types of measures (by additivity)

Type	Rule	Example
Additive	Can be summed across all dimensions	Revenue, units sold
Semi-additive	Summed across some dimensions but not time	Account balance, stock level
Non-additive	Cannot be summed; must be recalculated	Ratios, margin %, average price

Types of aggregate functions

Class	Meaning	Examples
Distributive	Sub-totals combine directly	SUM, COUNT, MIN, MAX
Algebraic	Computed from a few distributive values	AVG (SUM / COUNT)
Holistic	Need all the raw data	MEDIAN, RANK

This classification matters because distributive and algebraic functions can be rolled up from pre-computed aggregates, whereas holistic ones cannot.

Schemas that feed a cube

Schema	Description
Star	One fact table joined to denormalised dimension tables (simplest, fastest)
Snowflake	Dimensions are normalised into sub-tables (saves space, more joins)
Fact constellation (galaxy)	Several fact tables share dimensions, for example sales and inventory

Types of OLAP servers

Type	Storage	Strength	Weakness
MOLAP	Special multidimensional arrays	Fastest queries	Limited by cube size, extra storage, cube refresh time
ROLAP	Relational database (star schema), SQL generated on the fly	Scales to very large data, near real-time	Slower for complex queries
HOLAP	Summaries in cubes, detail in relational tables	Balance of speed and scale	More complex to manage

Query language. Relational OLAP uses SQL extensions (GROUP BY CUBE, ROLLUP, window functions). Multidimensional servers commonly use MDX (Multidimensional Expressions).

OLAP operations and multidimensional analysis

Operation	What it does	Example
Roll-up	Aggregate to a higher level of a hierarchy, or remove a dimension	Quarterly sales → yearly sales
Drill-down	Move to a more detailed level, or add a dimension	Year → quarter → month
Slice	Fix one dimension to a single value, giving a sub-cube	Revenue for 2025 only
Dice	Select ranges on two or more dimensions	North and South, 2024 and 2025
Pivot (rotate)	Swap the axes to see the data from another angle	Regions as rows ↔ years as rows
Drill-through	Go from a cube cell to the underlying detail rows	One cell → the individual transactions
Drill-across	Compare across cubes that share dimensions	Sales cube vs inventory cube
Ranking / top-N	Order members by a measure	Top 10 products by profit

How OLAP supports multidimensional analysis

1. The same measure can be viewed by any combination of dimensions without writing new queries.
2. Hierarchies let the user zoom in and out, from a company-wide figure down to a single store.
3. Analysts can compare dimensions side by side, for example region against time.
4. Slice and dice isolate the segment that matters, such as one product line in one region.
5. Pivoting changes the view in seconds, so exploration is interactive and iterative.
6. Calculated members (growth %, share of total) can be added without changing the stored data.
7. Drill-through links the summary to the detail, so users can check where a number came from.
8. Cross-dimensional comparisons expose patterns that single-dimension reports hide (for example a product that sells well overall but poorly in one region).

How OLAP supports data summarization

Summarization means turning detailed records into totals, averages, and counts. OLAP does this in several ways.

1. **Pre-aggregation.** Totals at every level are computed in advance, so a query for "annual revenue by region" reads a stored number instead of scanning millions of rows.
2. **The aggregation lattice.** With n dimensions and no hierarchies, a cube has 2^n cuboids. Three dimensions give $2^3 = 8$: the detailed combination, three pairs, three single dimensions, and the grand total.
3. **Hierarchies multiply the cuboids.** With hierarchies the count is the product of (levels + 1) across dimensions. For example, time with 4 levels (day, month, quarter, year), product with 2 (item, category), and region with 3 (store, city, region) give $5 \times 3 \times 4 = 60$ cuboids.
4. **Roll-up along hierarchies.** Summaries are produced at any level on demand.
5. **Aggregate functions.** SUM, COUNT, AVG, MIN, MAX, plus derived measures such as margin %.
6. **Materialisation strategies**

Strategy	**Description**	**Trade-off**
No materialisation	Compute everything at query time	Slow queries, no extra storage
Full materialisation	Pre-compute every cuboid	Fastest queries, heavy storage and load time
Partial materialisation	Pre-compute only the most useful cuboids	Balanced; the usual choice

7. **Supporting techniques.** Bitmap indexes, compression, partitioning, and caching speed up queries further.
8. **Sparsity.** Many cells are empty, so the cube stores only non-empty cells to save space.

Example. One year of sales may hold 50 million line items. The cube answers "Revenue by region and quarter" from a table of 16 numbers (4 regions × 4 quarters). The detail is still available through drill-through.

Trade-off. More pre-computed aggregates give faster queries but need more storage and longer refresh times. Designers choose which cuboids to materialise based on the questions users ask most often.

7. Constructing an OLAP cube: revenue by region and year

Step 1. Choose the business process and grain. The process is sales, and the grain is one row per sold line item.

Step 2. Identify dimensions and hierarchies

- Region: store → city → region → country.
- Year (Time): day → month → quarter → year.
- Product (optional third dimension, shown as the depth in the diagram): product → category.

Step 3. Identify the measure. Revenue, an additive measure aggregated with SUM.

Step 4. Source the data. It comes from fact_sales joined to dim_store, dim_date, and dim_product in the warehouse.

Step 5. Define the cube. Choose the storage mode (MOLAP, ROLAP, or HOLAP) and which aggregations to materialise.

Step 6. Build and aggregate. The cube is computed with GROUP BY CUBE, which produces every combination of totals:

```
SELECT s.region, d.year_num, SUM(f.sales_amount) AS revenue
FROM fact_sales f
JOIN dim_store s ON s.store_key = f.store_key
JOIN dim_date  d ON d.date_key  = f.date_key
GROUP BY CUBE (s.region, d.year_num);
```

Step 7. Validate and publish. Reconcile the cube totals with the warehouse, then expose the cube to dashboards and pivot tools.

Resulting two-dimensional view (all products, in millions)

Region	**2023**	**2024**	**2025**	**Region total**
North	40	46	55	141
South	30	35	41	106
East	22	25	30	77
West	33	38	44	115
Year total	**125**	**144**	**170**	**439**

Reading the cube

- A two-dimensional cube has 4 groupings: region and year, region only, year only, and the grand total (439).
- The year totals show growth of 15.2% (2023 to 2024) and 18.1% (2024 to 2025).
- Regional share of 2025 revenue: North 32.4%, South 24.1%, East 17.6%, West 25.9%.
- Regional growth 2024 to 2025: North 19.6%, South 17.1%, East 20.0%, West 15.8%. East is the smallest region but is growing fastest.

Design considerations

- Keep the grain consistent so facts do not double count.
- Use conformed dimensions if the cube will be compared with other cubes.
- Handle slowly changing dimensions (for example a store moving to another region) so history is not distorted.

- Include an "Unknown" member for missing dimension values.

Applying OLAP to quarterly sales data

Sample data: 2025 revenue by region and quarter (millions)

Region	Q1	Q2	Q3	Q4	Year
North	11	12	13	19	55
South	8	9	10	14	41
East	6	7	7	10	30
West	9	10	11	14	44
Total	**34**	**38**	**41**	**57**	**170**

Techniques applied

Technique	Application	Result
Roll-up	Quarters → year	Total 2025 revenue = 170
Drill-down	Open Q4 into October, November, December	Find which month drives the peak
Slice	Fix Q4	Q4 revenue by region: North 19, South 14, East 10, West 14
Dice	North and West only, Q3 and Q4	Focused comparison of the two strongest regions
Pivot	Quarters as rows, regions as columns	Compare regions at each quarter
Ranking	Order regions by Q4 revenue	North leads, East lags
Share of total	Q4 / year	57 / 170 = 33.5% of annual revenue falls in Q4
Drill-through	Open the North Q4 cell	See the individual transactions behind the 19

Period comparisons

Measure	Q1	Q2	Q3	Q4
2024 revenue (total)	30	33	36	45
2025 revenue (total)	34	38	41	57

Measure	Q1	Q2	Q3	Q4
Year-on-year growth	13.3%	15.2%	13.9%	26.7%
Quarter-on-quarter growth (2025)	–	11.8%	7.9%	39.0%

Time-intelligence measures

Measure	Calculation	Result
Year-to-date (to Q3, 2025)	34 + 38 + 41	113
Same period last year (to Q3, 2024)	30 + 33 + 36	99
YTD growth	113 / 99 − 1	14.1%
Rolling four-quarter sum at Q3 2025	45 + 34 + 38 + 41	158
Rolling four-quarter sum at Q4 2025	34 + 38 + 41 + 57	170
Variance against target (assumed Q4 target 52)	57 − 52	+5 (+9.6%)

Insights

- Revenue peaks in Q4, a seasonal pattern typical of retail (festivals, year-end promotions).
- Q4 growth (26.7% YoY) is well above the other quarters, which suggests the Q4 campaigns worked.
- East has the smallest share but the fastest annual growth, so it deserves a drill-down by store or product.
- Q2 and Q3 growth is steady at about 14 to 15%, so the Q4 jump is not simply a continuation of the underlying trend.

Suggested actions

- Plan stock and staffing for the Q4 peak in the second half of the year.
- Investigate what drove the 39% jump from Q3 to Q4, then repeat it in weaker quarters.
- Review the South and West quarters for promotion opportunities.

OLAP for forecasting and trend analysis in retail

OLAP does not forecast by itself. It prepares and presents the historical data, and simple statistical models or data mining tools produce the forecast from the cube's output.

How OLAP supports it

1. Provides clean, consistent, multi-year history by day, week, month, and quarter.
2. Lets analysts drill down to find where a trend starts, by product, store, or region.
3. Calculates growth rates, moving averages, and period-over-period change as measures.

4. Feeds the aggregated series into forecasting models (regression, exponential smoothing, ARIMA) or BI tools.
5. Supports what-if analysis, for example "what if growth slows to 10%?"
6. Lets analysts compare forecast against actual in the same cube.

Components of a time series

Component	Meaning	Retail example
Trend	Long-term direction	Annual revenue rising
Seasonality	Regular pattern within a year	Q4 peak
Cyclical	Longer swings tied to the economy	Spending falls in a downturn
Irregular	Random events	A strike or a weather event

Techniques

Technique	Method	Retail illustration
Year-on-year trend	Compare the same period across years	Annual revenue 125 → 144 → 170 (growth 15.2%, then 18.1%)
Moving average	Average of the last n periods, to smooth noise	A 4-quarter moving average removes seasonality from the trend line
Linear trend	Fit a straight line to the history	The 2023 to 2025 trend gives about 191 for 2026
Growth-rate projection	Apply the average growth rate	Average growth of about 16.6% gives about 198 for 2026
Exponential smoothing	Weight recent periods more heavily	With a weight of 0.5, the forecast for 2026 is about 152
Seasonal index	Share of each quarter in the annual total	Q4 is about 32% of the year (index about 1.3)
Seasonal naive	Last year's quarter scaled by growth	Q1 2026 ≈ 34 × 1.18 ≈ 40
Combined forecast	Annual forecast × seasonal share	2026 Q4 ≈ 191 × 32.4% ≈ 62

Note on the exponential smoothing result. The smoothed forecast (about 152) is far below the trend-based forecasts (191 to 198). This is a known weakness: simple exponential smoothing lags behind a series that is rising steadily. It shows why the method should match the pattern in the data.

Measuring forecast quality. Compare forecasts with actuals using error measures such as MAD (mean absolute deviation) or MAPE (mean absolute percentage error). Because the cube holds both forecast and actual values, the error can be reported by region, product, and quarter.

Retail uses

- Demand planning and inventory replenishment before peak seasons.
- Staffing and logistics planning.
- Setting sales targets and budgets by region.
- Detecting declining products or stores early for markdown or removal.
- Measuring the lift from a promotion against the normal trend.
- Planning cash flow and supplier orders.

Limitations

- Forecasts assume past patterns continue, so they should be adjusted for new stores, price changes, or economic shifts.
- Short histories (here only three years) give unreliable forecasts and weak seasonal indices.
- Forecasts at a very detailed level (one product in one store) are noisier than aggregated ones.

Business case: using OLAP to improve marketing performance

Organisation. An omnichannel retailer selling through stores and a website.

Problem

- Campaign results are reported monthly from spreadsheets, so they arrive too late to adjust spend.
- No single view of spend, orders, and revenue by channel, campaign, and customer segment.
- Marketing budget is spread evenly across channels instead of toward the best performers.
- Different teams report different numbers for the same campaign.
- No link between marketing activity and in-store sales.

Objectives

1. Measure return on marketing spend (ROMI) by campaign, channel, segment, and region.
2. Shift budget toward the highest-return activities.
3. Reduce customer acquisition cost (CAC).
4. Increase repeat purchases and customer lifetime value (CLV).
5. Shorten the time from campaign launch to performance insight, from weeks to days.

Solution: a marketing OLAP cube

Element	Design
Source data	CRM, e-commerce platform, POS, email and ad platforms, loyalty programme
Fact table	Campaign performance: spend, impressions, clicks, orders, revenue, new customers
Dimensions	Date, campaign, channel (email, social, search, in-store), customer segment, product category, region
Hierarchies	Day → week → month → quarter; campaign → channel; product → category
Calculated measures	ROMI = (revenue − spend) / spend; CAC = spend / new customers; conversion rate = orders / clicks; average order value; CLV

Key performance indicators

- Return on marketing investment (ROMI).
- Customer acquisition cost (CAC).
- Conversion rate and click-through rate.
- Average order value.
- Customer lifetime value.
- Repeat purchase rate and churn rate.
- Campaign lift against a baseline.

How OLAP is used

Marketing question	OLAP operation
Which channel gives the best ROMI this quarter?	Roll-up by channel, slice on the quarter
Why did a campaign underperform?	Drill-down by segment, region, and week
How do 18-to-25-year-old customers in the South respond to social ads?	Dice on segment, region, and channel
Are repeat buyers growing?	Trend by customer cohort over time
What drives Q4 revenue?	Compare the campaign mix across years
Which campaigns suit which products?	Pivot campaign against product category

Marketing question	OLAP operation
Which customers should receive the next offer?	Segment analysis by purchase history (recency, frequency, monetary value)

Illustrative channel analysis (assumed figures for the case, in millions)

Channel	Spend	Revenue attributed	ROMI
Email	2.0	10.0	4.0
Search	6.0	21.0	2.5
Social	7.0	17.5	1.5
Print and in-store	5.0	10.0	1.0
Total	**20.0**	**58.5**	**1.9**

What the analysis shows

- Email returns four times its spend, and print and in-store only returns its spend once.
- A part of the print and in-store budget could be moved to email and search.
- Returns diminish as spend rises, so a channel should not be funded without testing how much more it can absorb. The cube lets managers watch ROMI as spend changes.
- CAC can be compared in the same way. For instance, if social acquires 35,000 new customers for 7.0 million (CAC of 200), while email acquires 20,000 for 2.0 million (CAC of 100), email is the cheaper source of new customers.

Expected benefits

- Marketing decisions based on the latest data instead of month-old reports.
- Budget moved from low-ROMI to high-ROMI channels.
- Better targeting, since the segments that respond are identified.
- One agreed definition of campaign metrics.
- Less analyst time spent preparing reports.
- Faster testing of campaign ideas (A/B comparison by segment).

Illustrative return on investment (assumed figures for the case, not real data)

Item	Value
Annual marketing spend	20 million
Estimated extra margin per year from better allocation	4 million
First-year cost (software, build, training)	1.5 million

Item	Value
ROI	(4 − 1.5) / 1.5 ≈ 167%

Implementation roadmap

1. Define KPIs and agree metric definitions with marketing.
2. Extend the warehouse with campaign and channel data.
3. Build the marketing data mart and OLAP cube.
4. Create dashboards for campaign managers and executives.
5. Train users and pilot with one channel.
6. Review results, tune, and roll out to all channels.

Risks and mitigation

Risk	Mitigation
Poor data quality from ad platforms	Validation rules in the ETL, data stewardship
Customer matching across channels	Common customer key (email or loyalty ID)
Attribution is hard (which touchpoint gets the credit?)	Agree an attribution rule up front and document it in the metadata
Low user adoption	Training, simple dashboards, executive sponsor
Privacy and consent	Follow data protection rules and apply role-based access
Overfitting decisions to short-term data	Review trends over several periods before moving budget

Advantages and limitations of OLAP

Advantages

1. Fast answers to complex, aggregated queries.
2. Intuitive for business users, with little technical skill needed.
3. Consistent business definitions across the organisation.
4. Flexible exploration along any dimension and hierarchy.
5. Supports planning, budgeting, and forecasting.
6. Takes analytical load off operational systems.

Limitations

1. Cubes need to be designed up front, and new dimensions require rebuilding.
2. MOLAP cubes need extra storage and refresh time, and data may not be real time.
3. Large numbers of dimensions or very high-cardinality dimensions (for example millions of customers) make cubes huge and sparse.
4. Pre-computed aggregates can become stale between refreshes.
5. Quality depends on the warehouse: errors in the source data appear in the cube.
6. OLAP describes what happened. It does not explain why or predict outcomes without additional analytics.

Metadata in Data Warehousing:

Definition. Metadata is "data about data." It describes the content, structure, origin, meaning, quality, and use of the data in a warehouse. The warehouse holds the facts (for example, a sale of 2,400 on 11 October). Metadata explains what that number means, where it came from, how it was calculated, and when it was loaded.

An analogy. In a library, the books are the data. The catalogue card (title, author, shelf location, edition) is the metadata. Without the catalogue the books are still there, but nobody can find or trust them.

Why a warehouse needs more metadata than an operational system

1. Data comes from many sources with different names, formats, and meanings.
2. Data is transformed on the way in, so the stored value is not the source value.
3. It holds years of history, during which definitions and structures change.
4. Its users are mostly business analysts, not the programmers who built it.
5. Its numbers are used for decisions, so users must be able to verify them.

Metadata and the data dictionary. A data dictionary is a limited form of metadata. It lists tables, columns, data types, and descriptions. Warehouse metadata goes further by adding business definitions, lineage, transformation rules, load history, and usage.

Where metadata sits in the architecture

Sources → Extract → Staging → Transform → Data warehouse → Data marts / OLAP → Users

↓ ↓ ↓ ↓ ↓ ↓ ↓

METADATA REPOSITORY (central)

Layer	Metadata held about it
Source	System names, tables, file layouts, owners, extract methods, refresh schedules
ETL / staging	Mappings, transformation and cleansing rules, job dependencies, rejects
Warehouse	Schemas, keys, grain, dimensions, hierarchies, history rules (for example slowly changing dimensions)

Layer	Metadata held about it
Marts and OLAP	Cube definitions, measures, calculated members, aggregations
Delivery	Report and dashboard definitions, business names, access rights
Whole system	Load history, data quality results, usage, security, audit trail

Every component both reads metadata and writes it. This is why it is described as the "blueprint" or the "glue" of the warehouse.

The three main categories of metadata

#	Category	Describes	Main audience
1	**Business metadata**	What the data means in business terms	Business users, analysts
2	**Technical metadata**	How the data is structured, stored, and moved	Developers, DBAs, ETL engineers
3	**Operational metadata**	How the warehouse is running and how the data is used	Operations team, administrators, auditors

Alternative classifications you may meet

Source	Classification
Ralph Kimball	Technical, business, and process (back-room) metadata
Some textbooks	Back-room metadata (used by the ETL team) and front-room metadata (used by end users)
Some texts	Build-time vs run-time metadata

These map onto the three-way split. Back-room metadata is mostly technical and operational. Front-room metadata is mostly business. If your syllabus lists three categories, give business, technical, and operational.

Business metadata

Purpose. It lets non-technical users find, understand, and trust the data.

Typical contents

- Business glossary and definitions (for example "net sales = quantity × price − discount, excluding tax").
- Business rules and calculations for KPIs.
- Data ownership and stewardship (who is responsible for each subject area).
- Subject-area descriptions and data-quality expectations.
- Report and dashboard descriptions, and who they are for.

- Business names for technical fields (sales_amount is shown as "Net Sales").
- Data classification and sensitivity (public, internal, confidential).
- Hierarchy descriptions (store → city → region).
- Units, currencies, and time zones.
- Known limitations (for example "online returns are loaded weekly").
- Synonyms and related terms ("revenue" vs "turnover").
- Valid value lists (for example channel is "Store" or "Online").

Example from the retail warehouse

Item	**Business metadata**
Term	Net sales
Definition	Revenue after discounts and before tax
Owner	Finance department
Used in	Monthly sales dashboard
Refresh	Daily by 6 a.m.
Known limitation	Excludes gift-card sales

Why it matters. Most disputes about reports come from different meanings of the same word. Business metadata settles them in one place.

Technical metadata

Purpose. It describes the structure of the warehouse and the processes that build it. Developers and administrators use it to design, build, change, and maintain the system.

Typical contents

Area	**Examples**
Source descriptions	Source systems, tables, file layouts, connection details, extraction method
Data structures	Table and column names, data types, lengths, keys, indexes, constraints
Schema design	Star-schema relationships, fact and dimension definitions, grain, hierarchies
Source-to-target mappings	Which source field feeds which warehouse column
Transformation rules	Cleansing, derivations, lookups, data-type conversions

Area	Examples
ETL job design	Job names, dependencies, schedule, parameters
Data lineage	The path of a data item from source to report
Physical storage	Partitions, tablespaces, aggregates, and cube definitions
Security	Roles, permissions, and access rules
Versioning	Version numbers and change history of the design
Default and missing-value rules	For example "unknown customer" key of −1
History handling	How slowly changing dimensions are treated

Example: source-to-target mapping entries

Source	Target	Rule
stg.pos_sales.unit_price (TEXT)	dw.fact_sales.unit_price (NUMERIC(10,2))	Cast to numeric, reject negative values
stg.erp_products.sku	dw.dim_product.sku	Trim spaces, convert to upper case
stg.pos_sales.txn_ts	dw.fact_sales.date_key	Extract date, look up the key in dim_date
stg.crm_customers.state	dw.dim_customer.state	Standardise names ("KL" → "Kerala")

Why it matters. It is the written design of the system and the basis for maintenance and change.

Operational metadata

Purpose. It records what happens when the warehouse runs: loads, jobs, errors, and usage. It is generated automatically as the system operates.

Typical contents

Area	Examples
Load history	Start and end time of each load, status (success, failed, partial)
Volumes	Rows extracted, rows loaded, rows rejected
Errors and rejects	Error messages, rejected records and reasons
Job scheduling	Run frequency, dependencies, delays, restarts
Data freshness	Time of the last successful refresh per table

Area	Examples
Data-quality results	Null counts, duplicate counts, reconciliation checks
Usage statistics	Who queried what, how often, and how long queries took
Performance	Query run time, resource use, storage growth
Archive and purge history	What was archived, when, and where
Audit trail	Changes to data and to the structure
Capacity data	Table sizes and growth rates
Backup and recovery records	When backups ran and whether they succeeded

Example: one load-log record

Field	Value
Job	load_fact_sales
Run date	11 Oct 2026
Start / end	02:00 / 02:14
Rows read	1,250,000
Rows loaded	1,248,700
Rows rejected	1,300 (invalid dates)
Status	Completed with warnings

A simple reconciliation check built from operational metadata: rows read (1,250,000) − rows loaded (1,248,700) = 1,300, which equals the rows rejected. If the numbers did not balance, the load would be flagged for investigation.

Why it matters. It supports monitoring, scheduling, tuning, auditing, and capacity planning.

Comparison of the three categories

Aspect	Business	Technical	Operational
Question answered	What does it mean?	How is it built?	What happened, and how is it used?
Language	Business terms	Technical terms	Logs and statistics
Created by	Business users, data stewards	Developers, designers	The system itself
Rate of change	Low	Low to moderate	Constant

Aspect	Business	Technical	Operational
Main users	Analysts, managers	Developers, DBAs	Operations, administrators, auditors
Example	Definition of net sales	Column type and mapping rule	Load time and rows rejected

Distinguishing technical and operational metadata

Aspect	Technical metadata	Operational metadata
Core question	How is the data structured and built?	How is the system running and being used?
Nature	Design-time, relatively static	Run-time, constantly changing
Created by	Developers and designers (through design and ETL tools)	Generated automatically by the system as it runs
Changes when	The design or a rule changes	Every job run, load, or query
Volume	Moderate	Large and growing, because it logs every event
Content	Schemas, mappings, transformation rules, lineage	Load times, row counts, errors, usage, performance
Main users	Developers, DBAs, architects	Operations staff, administrators, auditors
Used for	Building, changing, and debugging the design	Monitoring, scheduling, tuning, and auditing
Retention	Kept for the life of the design, with versions	Kept for a set period, then summarised or archived
Example	unit_price is NUMERIC(10,2), mapped from the POS	The POS load on 11 Oct rejected 1,300 rows

Memory aid. Technical metadata is the *plan*. Operational metadata is the *log of what happened when the plan ran*. They are linked: the operational record "load rejected 1,300 rows" can only be understood by looking at the technical rule that rejected them.

Why metadata is important in data warehousing

1. **Makes the data understandable.** Business users can see definitions, units, and calculation rules, so they interpret figures correctly.

2. **Builds trust in the data.** Users trust a report when they can see where the data came from, how it was transformed, and when it was last refreshed.
3. **Supports data lineage and traceability.** Any figure in a report can be traced back through the warehouse to its source, which is vital for investigating errors and for audits.
4. **Drives ETL.** Mappings and transformation rules stored as metadata tell the ETL tools what to do. Where ETL is metadata-driven, a change to a rule means editing the metadata instead of rewriting code.
5. **Enables impact analysis.** Before a change, the team can see which tables, jobs, cubes, and reports it affects.
6. **Improves maintenance and reduces cost.** Documented structures and dependencies make the system easier to change, fix, and hand over to new staff.
7. **Ensures consistency.** One shared definition of each term and measure stops departments from using different meanings for the same word.
8. **Helps users find data.** A searchable catalogue lets users discover which tables and measures exist, rather than asking IT.
9. **Supports data quality management.** Quality rules, validation results, and reject counts are tracked as metadata.
10. **Supports governance, security, and compliance.** Metadata records ownership, sensitivity, access rules, and an audit trail, which supports regulations such as data-protection laws.
11. **Aids performance tuning.** Usage statistics show which tables and queries are used most, so indexes, partitions, and aggregates can be targeted.
12. **Supports integration.** It documents how data from different sources maps to a single warehouse model.
13. **Preserves knowledge.** It holds the design knowledge of the system independently of the people who built it.
14. **Enables self-service analytics.** With business names, descriptions, and sample values, users build their own reports with less IT help.
15. **Supports query optimisation.** Query tools and optimisers use metadata (statistics, indexes, aggregate tables) to choose efficient plans, and OLAP tools use it to rewrite queries against pre-built aggregates.
16. **Helps manage change over time.** Version history shows what a definition or structure was at a given date, so historical reports can be explained.
17. **Aids data migration and consolidation.** A complete inventory of objects and mappings speeds up moving to a new platform.
18. **Supports data retention and archiving decisions.** Usage metadata shows which data is no longer queried and could be archived.

Impact of missing technical metadata on system maintenance

Technical metadata is the written knowledge of how the warehouse is built. Without it, that knowledge exists only in people's memories and in the code.

Direct effects

Maintenance activity	What goes wrong without technical metadata
Impact analysis	No one can say what depends on a table or column, so every change is a risk.
Source changes	If a source renames or drops a field, the team cannot quickly find which ETL jobs, tables, and reports use it.
Debugging and troubleshooting	When a report number is wrong, the team cannot trace it back through the transformations, so root-cause analysis is slow.
Enhancements	Adding a column or a new source means first reverse-engineering the existing code to understand the current logic.
Staff changes and onboarding	New staff learn from reading code and asking colleagues. If a key person leaves, their knowledge leaves with them.
Testing	There is no reference for what the transformations should do, so regression tests are hard to write.
Migration and upgrades	Moving to a new platform needs a full inventory of objects, mappings, and dependencies, which has to be rebuilt from scratch.
Performance tuning	Without partition, index, and aggregate definitions on record, tuning decisions are guesswork.
Auditing and compliance	The organisation cannot demonstrate how a reported figure was derived.
Duplicate work	Developers cannot see that a transformation already exists, so they write another version with slightly different logic.
Release management	Changes cannot be compared with a documented baseline, so version control and rollback are unreliable.
Security reviews	Nobody can list who has access to which objects.
Dependency ordering	Job sequences and dependencies are unknown, so loads fail or run in the wrong order after a change.
Capacity planning	The team does not know which structures grow or why.

A worked example

The ERP team renames the column sku to item_code.

- **With technical metadata.** A lineage query shows that the column feeds two staging tables, one dimension, and the sales cube. The team updates exactly those objects and tests them.

- **Without it.** Nobody knows what uses sku. The nightly ETL fails or, worse, loads blanks silently. The sales dashboard shows missing products, and the team spends days searching through job code to find every reference.

Consequences over time

1. **Slower and costlier maintenance.** Every change begins with investigation.
2. **More errors.** Changes made without knowing the dependencies break other parts of the system.
3. **Reduced data quality and trust.** Hidden transformation errors produce wrong numbers that users cannot challenge.
4. **Dependence on key individuals.** The system becomes fragile and hard to hand over.
5. **Technical debt.** Undocumented, inconsistent logic builds up and eventually makes the warehouse too risky to change.
6. **Higher risk of project failure.** Warehouses that cannot be changed cannot keep up with the business.
7. **Longer outages.** Recovery after a failure takes longer because no one has the reference for how things should look.
8. **Compliance exposure.** The organisation may not be able to show regulators how a figure was produced.
9. **Loss of user confidence.** Users return to spreadsheets, and the warehouse loses its value as the single source of truth.

Mitigation

- Capture technical metadata automatically from ETL, modelling, and database tools instead of relying on manual documents.
- Store it in a central, searchable repository, and treat updating it as part of every change.
- Use lineage and impact-analysis tools.
- Include metadata review in the definition of "done" for each release.
- If metadata is already missing, rebuild it in priority order: the critical tables first, then mappings, then lineage.
- Use version control for all design artefacts.

Contrast: what happens when other metadata is missing

Missing	Main impact
Business metadata	Users misread or distrust the data, and departments report conflicting figures.
Technical metadata	Maintenance, change, and debugging become slow, risky, and person-dependent.

Missing	Main impact
Operational metadata	Failures go unnoticed, freshness is unknown, performance problems and misuse cannot be diagnosed, and audits fail.

Data lineage and impact analysis

Data lineage shows where a data item came from and what happened to it on the way.

Direction	Question answered	Use
Backward lineage	Where did this number come from?	Investigating errors, audit
Forward lineage	What uses this field?	Impact analysis before a change

Example lineage for "Net Sales" on a dashboard

POS system (unit_price, quantity, discount)

→ stg.pos_sales (raw text)

→ ETL job load_fact_sales (cast, validate, discount rule)

→ dw.fact_sales.sales_amount

→ Sales cube measure "Net Sales"

→ Monthly sales dashboard

Impact analysis uses forward lineage. If the discount rule changes, the repository lists the job, the fact column, the cube measure, and the dashboards that are affected, so they can be tested before release.

Levels of lineage: table level (which tables feed which), column level (which source columns feed a target column), and row level (which source records produced a specific row).

Metadata management

The metadata repository. A central store that collects metadata from all tools (ETL, modelling, BI, database) and makes it available to users. A shared repository avoids having metadata scattered across spreadsheets and documents.

Repository architectures

Approach	Description	Trade-off
Centralised	One repository holds all metadata	Single view, but needs heavy integration
Distributed (decentralised)	Each tool keeps its own metadata	Easy to start, but fragmented

Approach	Description	Trade-off
Federated	Tools keep their own metadata, and a layer links and searches them	A compromise

Passive vs active metadata

Passive	Active
Documents the system for people to read	Drives the system's behaviour (for example ETL generated from mapping metadata)
Can fall out of date	Stays accurate because the system depends on it

Key capabilities of a metadata solution

- Business glossary and data catalogue.
- Lineage, forward and backward.
- Impact analysis.
- Version control and change history.
- Search and browse for end users.
- Integration with ETL, modelling, and BI tools.
- Role-based access to the metadata itself.

Metadata lifecycle

1. **Capture** from tools, scans of databases, and manual input from stewards.
2. **Store** in the repository, with versions.
3. **Integrate** across sources, resolving differing names and meanings.
4. **Publish** to users through a catalogue or portal.
5. **Maintain** as the warehouse changes.
6. **Retire** obsolete definitions while keeping history.

Standards. The Common Warehouse Metamodel (CWM) from the Object Management Group is an industry standard for exchanging metadata between tools. XML-based interchange formats are commonly used.

Roles

Role	Responsibility
Data steward	Owns business definitions for a subject area
Metadata manager	Runs the repository and its standards

Role	Responsibility
Data architect	Maintains the technical design metadata
DBA and ETL team	Keep technical and operational metadata accurate
Business users	Use and give feedback on definitions

Common challenges

1. Metadata is spread across many tools in incompatible formats.
2. It becomes outdated if updating it is not part of normal work.
3. Business and technical users define terms differently.
4. Nobody owns it, so it is neglected.
5. The effort is underestimated and the benefit is hard to measure, so funding is cut.
6. Operational metadata grows quickly and needs retention rules.

Good practice

- Assign data stewards to business definitions.
- Automate collection wherever possible.
- Include metadata tasks in project plans.
- Tie metadata to governance and data-quality processes.
- Start with the most critical subject area, then expand.
- Make the catalogue easy for business users to search.

Metadata and OLAP

OLAP tools depend heavily on metadata.

- **Cube metadata** defines dimensions, hierarchies, measures, and calculated members.
- **Business names** let users pick "Net Sales" and "Region" from a menu instead of writing SQL.
- **Aggregation metadata** records which pre-computed summaries exist, so queries can be directed to them.
- **Security metadata** controls which members each role can see.

If this metadata is wrong or missing, users get wrong or unavailable results even when the underlying data is correct.

Definition and purpose of NoSQL

Definition. NoSQL ("Not Only SQL") is a family of database systems that store and retrieve data in ways other than the fixed rows and columns of a relational database. They typically have flexible schemas, scale out across many servers, and are designed for large volumes of varied data.

Purpose

1. Handle very large data volumes, such as clickstreams, sensor feeds, and social media content, by adding commodity servers (horizontal scaling) instead of buying one bigger machine.
2. Store semi-structured and unstructured data (JSON documents, text, graphs, logs) where every record may have different fields.
3. Give fast reads and writes at high concurrency, for example millions of users at once.
4. Allow rapid development, since the data model can change without a costly schema migration.
5. Provide high availability by replicating data across servers and data centres.
6. Match the data model to the application: documents for content, key-value for caches, graphs for relationships, columns for analytics.

Why it emerged. Web-scale companies (Google, Amazon, Facebook) found that a single relational server could not keep up with their data volume, velocity, and variety (the "three Vs" of big data). Systems such as Google's Bigtable and Amazon's Dynamo inspired the NoSQL movement.

Types of NoSQL databases

Document stores. Data is kept as self-describing documents, usually JSON or a binary form of it. Each document can have its own structure. Examples: MongoDB, Couchbase. Suited to catalogues, user profiles, and content management.

Key-value stores. Each item is a key with an opaque value, like a giant dictionary. Very fast lookups. Examples: Redis, DynamoDB, Riak. Suited to caching, sessions, and shopping carts.

Wide-column (column-family) stores. Data is organised into rows identified by a key, each holding flexible sets of column families. Examples: Cassandra, HBase, Google Bigtable. Suited to time-series data, logging, and very large write-heavy workloads.

Graph databases. Data is stored as nodes and relationships, so connected data can be traversed quickly. Examples: Neo4j, Amazon Neptune. Suited to social networks, recommendations, and fraud detection.

CAP theorem and BASE

CAP theorem (Eric Brewer). A distributed data store can guarantee at most two of three properties at the same time:

- **Consistency:** every read sees the latest write.
- **Availability:** every request receives a response.
- **Partition tolerance:** the system keeps working when network failures split the servers.

Network partitions cannot be avoided in a distributed system, so the real choice during a partition is between consistency and availability. Systems that favour consistency (for example MongoDB in its default configuration, HBase) may refuse some requests during a partition. Systems that favour availability (for example Cassandra, DynamoDB) keep answering, possibly with slightly stale data.

BASE versus ACID. Relational databases follow ACID: Atomicity, Consistency, Isolation, Durability. Many NoSQL systems follow BASE:

- **Basically Available:** the system stays up even if parts fail.
- **Soft state:** the data may change over time as replicas synchronise.
- **Eventual consistency:** if no new updates arrive, all replicas eventually hold the same value.

Modern NoSQL systems are not purely one or the other. MongoDB, for instance, supports multi-document ACID transactions and tunable consistency settings.

MongoDB: core concepts

MongoDB is the most widely used document database. It stores data as BSON (Binary JSON), which extends JSON with extra types such as dates and binary data.

Terminology compared with relational databases

- Database is similar to a database.
- Collection is similar to a table.
- Document is similar to a row.
- Field is similar to a column.
- Index is similar to an index.
- Embedded documents and references take the place of joins.
- _id is the primary key, added automatically if you do not supply it.

Key properties

- **Schema-flexible.** Documents in one collection can differ, although in practice most collections follow a common shape. Schema validation rules can be added when structure must be enforced.
- **Rich query language.** Filtering, projection, sorting, aggregation pipelines, text search, and geospatial queries.
- **Replication.** A replica set keeps copies of the data on several servers, with automatic failover to a new primary.
- **Sharding.** Data is split across servers by a shard key for horizontal scaling.
- **Indexing.** Single-field, compound, multikey (for arrays), text, geospatial, and TTL indexes.
- **Document size limit.** A single document can be up to 16 MB.

Common MongoDB data types

- **String:** UTF-8 text, for example a name.
- **Int32 and Int64 (Long):** 32-bit and 64-bit integers.
- **Double:** 64-bit floating-point number.
- **Decimal128:** high-precision decimal, recommended for money.
- **Boolean:** true or false.
- **Date:** a date and time stored as milliseconds since 1 January 1970 UTC.
- **Timestamp:** an internal type used by MongoDB for replication and sharding operations.
- **ObjectId:** a 12-byte unique identifier, the default type of _id. It encodes a creation time, a random value, and a counter.
- **Array:** an ordered list of values, which can hold mixed types.
- **Object (embedded document):** a document nested inside another.
- **Null:** a null value.
- **Binary data (BinData):** raw bytes, such as a file fragment or a UUID.
- **Regular expression:** a pattern stored as a value and used in queries.
- **JavaScript:** code stored as a value (rarely used).
- **MinKey and MaxKey:** special values that compare lower or higher than all others, used internally and in sharding.
- **Deprecated types:** Undefined, Symbol, and DBPointer should not be used in new designs.

Practical notes

- Use Decimal128 or an integer number of paise or cents for currency, because Double can introduce rounding errors.
- Store dates as the Date type, not as strings, so that range queries and date functions work.

The document in MongoDB

Definition. A document is the basic unit of data in MongoDB: a set of field-and-value pairs, written in a JSON-like format and stored as BSON. It is analogous to a row, but it can hold nested objects and arrays, so data that would need several related tables in a relational design can live in one document.

Characteristics

- Field names are strings and are unique within a document.
- Values can be any BSON type, including other documents and arrays.
- Every document has a unique _id within its collection.
- Documents in the same collection do not have to have identical fields.
- Related data can be embedded in the document, so one read retrieves everything the application needs.

- Field order is preserved.

Why it matters. The document model matches the objects used in application code, so little translation is needed between the program and the database. It also keeps related data together on disk, which speeds up reads.

Collections. Documents are grouped into collections. A collection is created automatically when the first document is inserted.

Storing student details as a MongoDB document (JSON)

```
{
  "_id": ObjectId("652f1a9e8b3c4d0012ab34cd"),
  "studentId": "S1001",
  "name": { "first": "Anjali", "last": "Menon" },
  "dateOfBirth": ISODate("2003-05-14T00:00:00Z"),
  "email": "anjali.menon@example.com",
  "department": "Computer Science",
  "semester": 5,
  "cgpa": 8.7,
  "isActive": true,
  "address": {
    "city": "Malappuram",
    "state": "Kerala",
    "pin": "676505"
  },
  "phones": ["+91-9000000001", "+91-9000000002"],
  "courses": [
    { "code": "CS501", "title": "Data Warehousing", "grade": "A" },
    { "code": "CS502", "title": "Machine Learning", "grade": "B+" }
  ]
}
```

Design points

- The name and address are embedded objects, so related fields stay together.
- phones is an array of values, since a student may have several numbers.

- courses is an array of embedded documents, so a student's courses are read in a single query.
- dateOfBirth uses the Date type, cgpa uses a number, and isActive uses a boolean.
- A second student could omit phones or add a scholarship field without any change to the collection.

Inserting it

db.students.insertOne({ studentId: "S1001", name: { first: "Anjali", last: "Menon" }, department: "Computer Science", semester: 5, cgpa: 8.7 })

Several documents at once use insertMany([...]).

Retrieving data using a specific field value

By one field

db.students.find({ department: "Computer Science" })

This returns every student document whose department equals "Computer Science".

One document only

db.students.findOne({ studentId: "S1001" })

Choosing which fields to return (projection). 1 includes a field, 0 excludes it.

db.students.find(

{ department: "Computer Science" },

{ _id: 0, studentId: 1, "name.first": 1, cgpa: 1 }

)

Sample result:

{ "studentId": "S1001", "name": { "first": "Anjali" }, "cgpa": 8.7 }

By a field inside an embedded document. Use dot notation, in quotes.

db.students.find({ "address.city": "Malappuram" })

By a value inside an array of documents

db.students.find({ "courses.code": "CS501" })

By a value in a simple array

db.students.find({ phones: "+91-9000000001" })

Using comparison and logical operators

db.students.find({ cgpa: { $gte: 8.0 } })

db.students.find({ department: { $in: ["Computer Science", "Mathematics"] } })

db.students.find({ $and: [{ semester: 5 }, { isActive: true }] })

db.students.find({ $or: [{ cgpa: { $gt: 9 } }, { semester: 7 }] })

Sorting, limiting, and counting

db.students.find({ department: "Computer Science" }).sort({ cgpa: -1 }).limit(5)

db.students.countDocuments({ department: "Computer Science" })

Speeding up the lookup. Create an index on the searched field so MongoDB does not scan the whole collection.

db.students.createIndex({ department: 1 })

Commands to create and query a MongoDB database

The following uses mongosh, the MongoDB shell.

```
// 1. Create / switch to a database (it is created when data is first stored)
use collegeDB

// 2. Create a collection explicitly (optional)
db.createCollection("students")

// 3. Insert documents
db.students.insertMany([
  { studentId: "S1001", name: "Anjali Menon", department: "Computer Science", semester: 5, cgpa: 8.7 },
  { studentId: "S1002", name: "Rahul Nair",   department: "Mathematics",      semester: 3, cgpa: 7.9 },
  { studentId: "S1003", name: "Fathima K",    department: "Computer Science", semester: 5, cgpa: 9.1 }
])

// 4. Query
db.students.find({ department: "Computer Science" })
db.students.find({ cgpa: { $gt: 8.5 } }, { _id: 0, name: 1, cgpa: 1 })

// 5. Update
db.students.updateOne({ studentId: "S1002" }, { $set: { cgpa: 8.1 } })

// 6. Delete
```

```
db.students.deleteOne({ studentId: "S1002" })

// 7. Index
db.students.createIndex({ studentId: 1 }, { unique: true })

// 8. Aggregation: average CGPA per department
db.students.aggregate([
  { $group: { _id: "$department", avgCgpa: { $avg: "$cgpa" }, students: { $sum: 1 } } }
])

// 9. Housekeeping
show dbs
show collections
db.dropDatabase()
```

Explanations

- use collegeDB selects the database. MongoDB does not create it on disk until a collection receives data.
- $set changes only the named field and leaves the rest of the document untouched.
- aggregate runs a pipeline of stages. $group here collects documents by department and computes the average.
- Optional validation can enforce structure:

```
db.createCollection("students", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["studentId", "name"],
    properties: { cgpa: { bsonType: "double", minimum: 0, maximum: 10 } }
  } }
})
```

Schema design in MongoDB: embedding and referencing

Because there are no joins at the heart of the model, the central design question is how to place related data.

Embedding. Put related data inside the parent document, as with address and courses in the student example.

- Best when the data is read together, belongs to one parent, and has a bounded size.
- Gives fast reads with one query and atomic updates of a single document.
- Risk: documents that grow without limit approach the 16 MB cap and slow down.

Referencing. Store the _id of one document inside another, like a foreign key, and look it up when needed ($lookup in aggregation).

- Best when the related data is large, shared by many parents, or changes often.
- Avoids duplication.
- Cost: more than one query or a $lookup.

Design guidelines

1. Model the data around the queries the application runs most often.
2. Embed data that is read together and has a one-to-few relationship.
3. Reference data with one-to-many (very many) or many-to-many relationships.
4. Accept controlled duplication (denormalisation) of rarely changing values for read speed.
5. Avoid unbounded arrays. Move them to their own collection.
6. Index every field used in frequent filters and sorts.

Applying the NoSQL model to a social media storage system

Requirements. Users create profiles, publish posts with images and hashtags, follow other users, like and comment, send messages, and see a timeline. The system must serve millions of users, absorb heavy writes, and let the schema evolve as new features appear.

Why a document database fits

- Posts, profiles, and comments are naturally document-shaped and vary in structure (text, photo, video, poll).
- Heavy read and write traffic requires horizontal scaling.
- New features can add fields without migrations.

Collections and example documents

Users:


```
{
  "_id": "u101",
  "username": "anjali_m",
  "displayName": "Anjali Menon",
  "bio": "Data nerd",
```

"joinedAt": ISODate("2024-03-02T09:00:00Z"),

"followerCount": 1240,

"followingCount": 310,

"settings": { "privacy": "public", "language": "en" }

}

Posts (comments stored separately, because they grow without limit):

{

"_id": "p9001",

"authorId": "u101",

"text": "Finished my data warehouse notes!",

"media": [{ "type": "image", "url": "https://cdn.example.com/i/55.jpg" }],

"hashtags": ["datawarehouse", "study"],

"createdAt": ISODate("2026-10-11T08:30:00Z"),

"likeCount": 87,

"commentCount": 12,

"location": { "type": "Point", "coordinates": [76.05, 11.05] }

}

Comments (referencing the post and author):

{ "_id": "c5001", "postId": "p9001", "authorId": "u102",

"text": "Great work!", "createdAt": ISODate("2026-10-11T08:45:00Z") }

Follows (one document per relationship, so a single user can follow millions without a huge array):

{ "_id": "f77", "followerId": "u102", "followeeId": "u101", "since": ISODate("2025-01-10T00:00:00Z") }

Likes: { postId, userId, createdAt } with a unique index on (postId, userId) to prevent double likes.

Messages: { conversationId, senderId, text, sentAt }, in a separate collection, shardable by conversationId.

Design decisions

- **Embedded:** media list, hashtags, settings, and location, because they are small and read with the post.
- **Referenced:** comments, likes, and follows, because they are unbounded and queried independently.
- **Denormalised counters:** likeCount, commentCount, followerCount are stored in the parent so the timeline does not need to count millions of rows. They are updated with $inc.

- **Indexes:**
    - posts on (authorId, createdAt desc) for a profile page,
    - posts on hashtags (a multikey index) for hashtag pages,
    - posts on a text index for search,
    - a 2dsphere index on location for nearby posts,
    - follows on (followerId) and (followeeId),
    - a TTL index on temporary data such as stories that expire after 24 hours.
- **Timeline generation.**
    - *Fan-out on read:* fetch the list of followees, then query their latest posts. It is simple, but slow for users who follow many accounts.
    - *Fan-out on write:* when a user posts, push the post id into a timeline document for each follower. Reads are fast, writes are heavier. Large accounts (celebrities) are usually handled with a hybrid of the two.
- **Sharding.** Shard posts by authorId (or a hashed id) to spread load. Shard messages by conversationId.
- **Replication.** Replica sets give failover and let reads of non-critical data go to secondary servers.
- **Caching.** A key-value store such as Redis holds hot timelines, sessions, and counters.
- **Polyglot persistence.** A graph database such as Neo4j can be added for "people you may know" recommendations, and a search engine for full-text search. Media files themselves live in object storage, with only their URLs in MongoDB.

Sample queries

```
// Latest 20 posts of a user
db.posts.find({ authorId: "u101" }).sort({ createdAt: -1 }).limit(20)


// Posts with a hashtag
db.posts.find({ hashtags: "datawarehouse" }).sort({ createdAt: -1 })


// Add a like and update the counter
db.likes.insertOne({ postId: "p9001", userId: "u102", createdAt: new Date() })
db.posts.updateOne({ _id: "p9001" }, { $inc: { likeCount: 1 } })


// Timeline (fan-out on read)
```

```
const ids = db.follows.find({ followerId: "u102" }).map(f => f.followeeId)

db.posts.find({ authorId: { $in: ids } }).sort({ createdAt: -1 }).limit(50)
```

Trade-offs

- Eventual consistency of counters is acceptable for likes but not for payments, so such features would use a relational or strongly consistent store.
- Denormalised data must be kept in step when the source changes (for example a changed display name copied into posts).

Column-based storage for analytics

Idea. A row store keeps all the values of one record together. A column store keeps all the values of one column together. For analytics, which usually reads a few columns across millions of rows, the column layout reads far less data.

Example data: a small sales table

id | date | region | product | amount

1 | 2026-01-05 | North | Laptop | 800

2 | 2026-01-05 | South | Phone | 500

3 | 2026-01-06 | North | Phone | 450

4 | 2026-01-06 | North | Laptop | 900

5 | 2026-01-07 | South | Laptop | 850

Row-oriented layout on disk (each record stored together):

[1, 2026-01-05, North, Laptop, 800] [2, 2026-01-05, South, Phone, 500] [3, ...] ...

Column-oriented layout on disk (each column stored together):

id: [1, 2, 3, 4, 5]

date: [2026-01-05, 2026-01-05, 2026-01-06, 2026-01-06, 2026-01-07]

region: [North, South, North, North, South]

product: [Laptop, Phone, Phone, Laptop, Laptop]

amount: [800, 500, 450, 900, 850]

Analytical query

```
SELECT SUM(amount) FROM sales WHERE region = 'North';
```

- A row store reads every complete row to find the region and amount.
- A column store reads only the region and amount columns. It finds positions 1, 3, and 4 where region is North and adds amounts 800 + 450 + 900 = 2,150. It never touches id, date, or product.

Why it suits analytics

1. **Less data read.** Queries touch only the columns they need, which matters in tables with hundreds of columns.
2. **Better compression.** Values within one column are similar, so they compress well. If the table is sorted by region, run-length encoding turns the column into "North × 3, South × 2". Dictionary encoding and bit-packing add further savings.
3. **Fast aggregation.** SUM, AVG, COUNT, and GROUP BY run over a tight array of numbers and can use vectorised (SIMD) processing.
4. **Good for scans and OLAP.** The pattern of few columns over very many rows is what dashboards and reports do.

Weaknesses. Reading or writing a single whole record is slower, since it must be reassembled from several columns. Frequent small updates are costly. For this reason column stores are used for analytics (OLAP) and row stores for transactions (OLTP).

Two things called "column-based"

- **Columnar analytical databases and file formats** (ClickHouse, Amazon Redshift, Google BigQuery, Apache Parquet, ORC) store each column separately for fast analytics, as shown above.
- **Wide-column NoSQL stores** (Cassandra, HBase, Bigtable) are keyed by row and group columns into column families. They are designed for massive scale and write-heavy workloads, with queries designed around the partition key.

A wide-column example (Cassandra CQL), designed for the query "sales by region and day":

```
CREATE TABLE sales_by_region (
  region text,
  sale_date date,
  sale_id int,
  product text,
  amount decimal,
  PRIMARY KEY ((region), sale_date, sale_id)
);


SELECT sale_date, SUM(amount) FROM sales_by_region
WHERE region = 'North' AND sale_date >= '2026-01-05' GROUP BY sale_date;
```

Here region is the partition key (it decides which server holds the data) and sale_date and sale_id order rows within the partition.

Drill-down and roll-up for analysis

Definitions

- **Roll-up** moves from detailed data to a more summarised level, by climbing a hierarchy (month → quarter → year) or by removing a dimension.
- **Drill-down** is the reverse: moving from a summary to finer detail (year → quarter → month → day), or adding a dimension.

These are OLAP operations. They work on the hierarchies of a dimension, so they apply in any store that can aggregate: a data warehouse, a column store, or MongoDB's aggregation pipeline.

Example data. A sales collection with one document per transaction:

{ "date": ISODate("2025-11-14T00:00:00Z"), "region": "North", "category": "Electronics", "amount": 1200 }

Totals used in the example (in millions, sample figures consistent with the earlier OLAP notes): 2025 total 170; Q1 34, Q2 38, Q3 41, Q4 57; within Q4: October 16, November 19, December 22.

Level 1: year (the roll-up result)

```
db.sales.aggregate([
  { $group: { _id: { $year: "$date" }, revenue: { $sum: "$amount" } } },
  { $sort: { _id: 1 } }
])
```

Result for 2025: 170.

Level 2: drill down from year to quarter

```
db.sales.aggregate([
  { $match: { date: { $gte: ISODate("2025-01-01"), $lt: ISODate("2026-01-01") } } },
  { $group: {
      _id: { $ceil: { $divide: [ { $month: "$date" }, 3 ] } },
      revenue: { $sum: "$amount" } } },
  { $sort: { _id: 1 } }
])
```

Result: Q1 34, Q2 38, Q3 41, Q4 57. Q4 stands out as the peak.

Level 3: drill down from Q4 to month

```
db.sales.aggregate([
  { $match: { date: { $gte: ISODate("2025-10-01"), $lt: ISODate("2026-01-01") } } },
  { $group: { _id: { $month: "$date" }, revenue: { $sum: "$amount" } } },
  { $sort: { _id: 1 } }
])
```

Result: October 16, November 19, December 22. December is the strongest month.

Level 4: drill down by another dimension (region within December)

```
db.sales.aggregate([
  { $match: { date: { $gte: ISODate("2025-12-01"), $lt: ISODate("2026-01-01") } } },
  { $group: { _id: "$region", revenue: { $sum: "$amount" } } }
])
```

Roll-up in the other direction. Running the queries in the reverse order (month → quarter → year) is a roll-up. Removing a dimension, for example adding up all regions into one total, is also a roll-up.

SQL equivalent. A relational or columnar warehouse produces all the levels in one statement:

```
SELECT year, quarter, month, SUM(amount)
FROM sales
GROUP BY ROLLUP (year, quarter, month);
```

This returns the month-level rows, quarter subtotals, year subtotals, and a grand total.

How the analysis proceeds

1. Start at year level and see that 2025 revenue is 170.
2. Drill down to quarters and notice that Q4 is far above the others.
3. Drill down to months and find that December carries most of the Q4 peak.
4. Drill down by region or category to see which segment drives December.
5. Roll back up to confirm that the pieces add to the 170 total.

Connection to column stores. A drill-down query reads only the date, region, and amount columns. A columnar engine is well suited to running these repeated group-by queries quickly. In MongoDB, indexes on date and pre-aggregated summary collections can speed up frequent roll-ups.

Advantages of NoSQL over RDBMS

1. **Flexible schema.** New fields can be added without altering tables. This suits fast-changing requirements and varied data.
2. **Horizontal scalability.** Data is spread across many inexpensive servers by sharding. A relational database usually scales up on one bigger machine, which is costlier and has a ceiling.
3. **High performance at scale.** Data that is read together is stored together (embedded documents), so many queries avoid joins. Write throughput can be very high.
4. **High availability.** Built-in replication and automatic failover keep the system running if a server fails.
5. **Handles varied data.** Documents, graphs, key-value pairs, and wide columns suit data that does not fit neatly into rows and columns.

6. **Developer productivity.** Documents map directly to objects in application code, so less mapping code is needed, and iteration is quicker.
7. **Geographic distribution.** Data can be placed close to users in different regions.
8. **Cost.** Open-source options running on commodity hardware or cloud instances reduce licence and hardware cost.
9. **Big data and real-time use.** Suited to streaming data, logs, IoT, and personalisation workloads.
10. **Specialised models.** The right type can be chosen for each job, for example a graph database for relationships.

Limitations and when an RDBMS is the better choice

- Complex multi-table joins and ad hoc reporting are easier in SQL.
- Strong ACID guarantees across many records are a core strength of relational systems (although MongoDB now offers multi-document transactions).
- Many NoSQL systems favour eventual consistency, which is unsuitable for banking or inventory counts.
- Data duplication from denormalisation must be managed.
- There is no single standard query language, and skills and tooling are less uniform.
- Mature relational systems have decades of tooling, standards, and expertise.

Practical conclusion. The two are complements. Many systems use polyglot persistence, with an RDBMS for transactions and finance and NoSQL for content, caching, and analytics.